

EDA (Exploratory Data Analysis)

Exploratory Data Analysis (EDA) merupakan proses analisis awal yang dilakukan untuk memahami dataset yang dimiliki sebelum menjalankan model statistik atau melakukan pengambilan keputusan.

Deskripsi Data: EDA dimulai dengan meringkas karakteristik utama dari dataset yang termasuk statistik deskriptif seperti mean, median, modus, kuartil, dan sebagainya. Ini membantu dalam pemahaman awal tentang distribusi dan struktur data.

Visualisasi Data: Penggunaan grafik dan plot adalah bagian integral dari EDA. Visualisasi memungkinkan untuk melihat pola, tren, dan anomali dalam data dengan cara yang lebih intuitif.

Identifikasi Anomali: EDA membantu dalam mendeteksi nilai-nilai anomali dalam dataset.

Pemrosesan Data: Dalam proses EDA, seringkali diperlukan untuk membersihkan dan mempersiapkan data sebelum analisis lebih lanjut.

Pemeriksaan Korelasi: EDA membantu dalam memahami hubungan antara variabel dalam dataset.

Pemilihan Fitur: Melalui EDA, dapat dipilih fitur-fitur yang paling relevan atau memiliki pengaruh signifikan terhadap variabel target.

Eksplorasi Lebih Lanjut: EDA dapat menjadi titik awal untuk pertanyaan lebih lanjut dan penelitian lanjutan. Analisis awal dapat mengungkap pola atau tren yang menarik, yang kemudian dapat diinvestigasi lebih lanjut untuk mendapatkan pemahaman yang lebih dalam tentang data.

Interpretasi Hasil: Selain dari mempersiapkan data untuk analisis, EDA juga membantu dalam menafsirkan hasil dari analisis yang dilakukan. Ini membantu pengambilan keputusan yang lebih baik berdasarkan pemahaman yang mendalam tentang data yang ada.

Read dataset

Studi kasus : Credit Card **customers**

dataset dapat didownload pada link dibawah ini <https://www.kaggle.com/datasets/sakshigoyal7/credit-card-customers>

Tugas 1

1. Download dataset diatas, kemudian simpan di google drive anda atau github anda!
2. Silahkan membaca dataset diatas menggunakan dataframe!
3. Jelaskan apa tujuan dari penggunaan dataset ini!
4. Definisikan attribute mana yang menjadi input dan attribute mana yang menjadi output/ class/ label!
5. Berikan penjelasan singkat untuk setiap attribute/ variable dari dataset tersebut

```
In [1]: import pandas as pd
df_churning = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/BankChurners.csv', delimiter=',')

#fungsi head digunakan dataframe untuk melihat baris2 atas
df_churning.head()

# sampai di step ini kita menggunakan data structure/ struktur data dataframe dari pandas
# selain dataframe, di pandas ada struktur data yaitu series
# dataframe untuk 2 dimensi, series untuk 1 dimensi
# https://www.educative.io/answers/series-vs-dataframe-in-pandas
```

Out[1]:

	CLIENTNUM	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category	Months_on_book	...	Months_Inactive_12_mon	Contacts_Count
0	768805383	Existing Customer	45	M	3	High School	Married	\$60K - \$80K	Blue	39	...	1	
1	818770008	Existing Customer	49	F	5	Graduate	Single	Less than \$40K	Blue	44	...	1	
2	713982108	Existing Customer	51	M	3	Graduate	Married	\$80K - \$120K	Blue	36	...	1	
3	769911858	Existing Customer	40	F	4	High School	Unknown	Less than \$40K	Blue	34	...	4	
4	709106358	Existing Customer	40	M	3	Uneducated	Married	\$60K - \$80K	Blue	21	...	1	

5 rows x 21 columns

```
In [2]: #fungsi tail untuk melihat baris2 bawah

df_churning.tail()
```

Out[2]:

	CLIENTNUM	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category	Months_on_book	...	Months_Inactive_12_mon	Contacts_Count
10122	772366833	Existing Customer	50	M	2	Graduate	Single	\$40K - \$60K	Blue	40	...	2	
10123	710638233	Attrited Customer	41	M	2	Unknown	Divorced	\$40K - \$60K	Blue	25	...	2	
10124	716506083	Attrited Customer	44	F	1	High School	Married	Less than \$40K	Blue	36	...	3	
10125	717406983	Attrited Customer	30	M	2	Graduate	Unknown	\$40K - \$60K	Blue	36	...	3	
10126	714337233	Attrited Customer	43	F	2	Graduate	Married	Less than \$40K	Silver	25	...	2	

5 rows x 21 columns

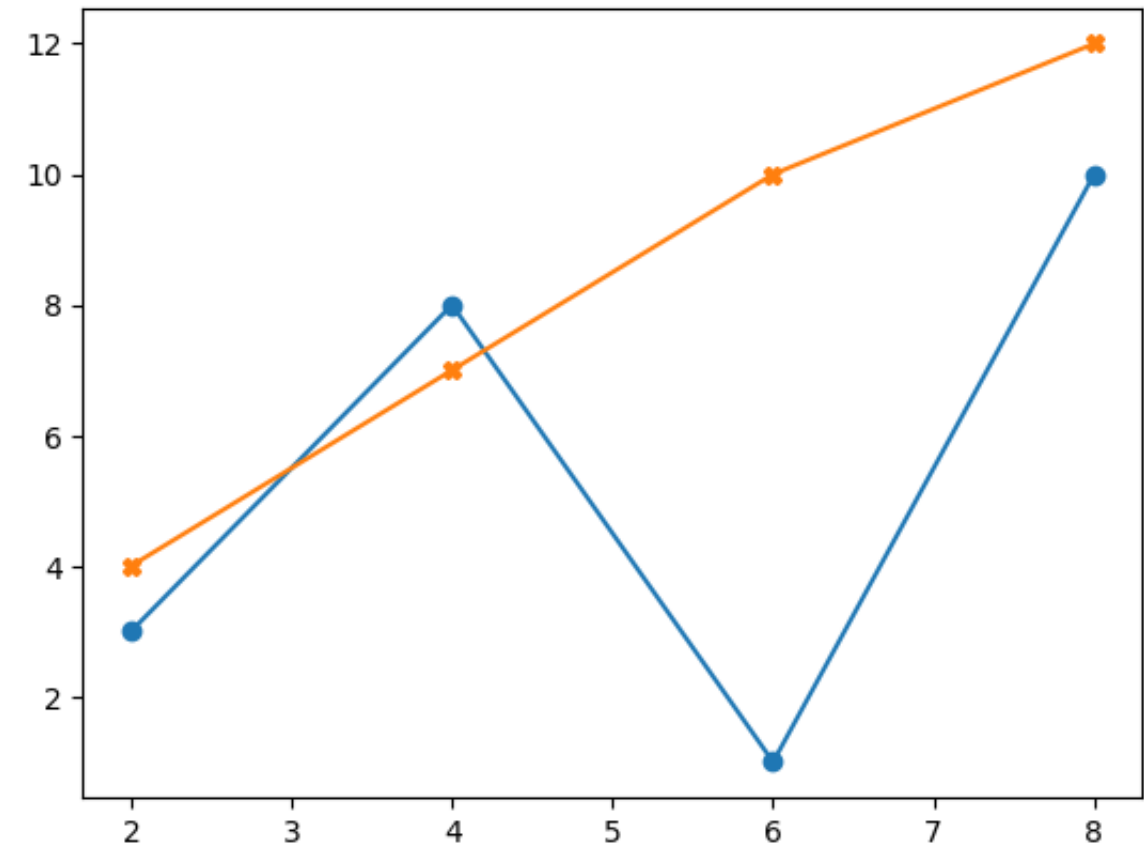
```
In [3]: existing_data = df_churning[(df_churning.Attrition_Flag == 'Existing Customer')]  
        attrited_data = df_churning[(df_churning.Attrition_Flag == 'Attrited Customer')]  
  
# menyimpan nilai dari class menjadi 2 dataframe yang berbeda
```

Contoh visualisasi

line chart

Line chart adalah jenis grafik yang menampilkan hubungan atau tren antara dua variabel atau lebih melalui penggunaan garis yang menghubungkan titik-titik data pada sumbu x dan y.

```
In [4]: # secara default fungsi plot() akan menampilkan garis/ line chart  
        # menggunakan library matplotlib untuk visualisasi  
        import matplotlib.pyplot as plt  
        import numpy as np  
  
        #sumbu x  
        x = np.array([2, 4, 6, 8])  
  
        #sumbu y  
        y1 = np.array([3, 8, 1, 10])  
        y2 = np.array([4, 7, 10, 12])  
  
        plt.plot(x, y1, marker = 'o')  
        plt.plot(x, y2, marker = 'X')  
  
        plt.show()  
  
        #silahkan pelajari lebih lanjut fungsi plot dari dokumentasi resmi dibawah ini  
        #https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html  
  
        # silahkan modifikasi data dari plot diatas
```



```
In [5]: # contoh menggunakan dataset sensor. Occupancy detection pertemuan sebelumnya
# baca dataset dari google drive anda, atau dari link github anda

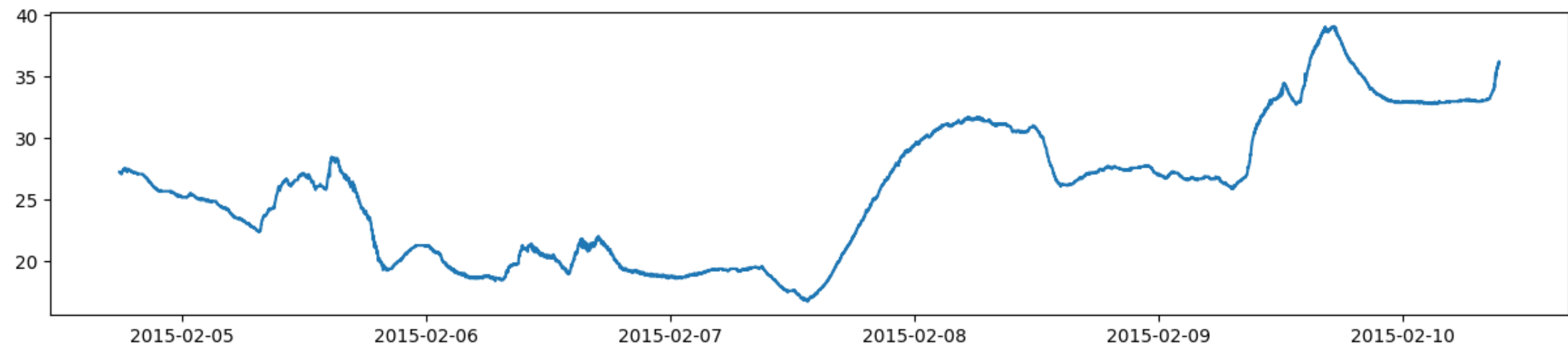
import pandas as pd
df_iot = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/datatraining.txt', delimiter=',')
#df_iot.head()
df_iot['date'] = pd.to_datetime(df_iot['date'])
```

```
In [6]: df_iot.head()
```

Out[6]:

	date	Temperature	Humidity	Light	CO2	HumidityRatio	Occupancy
1	2015-02-04 17:51:00	23.18	27.2720	426.0	721.25	0.004793	1
2	2015-02-04 17:51:59	23.15	27.2675	429.5	714.00	0.004783	1
3	2015-02-04 17:53:00	23.15	27.2450	426.0	713.50	0.004779	1
4	2015-02-04 17:54:00	23.15	27.2000	426.0	708.25	0.004772	1
5	2015-02-04 17:55:00	23.10	27.2000	426.0	704.50	0.004757	1

```
In [7]: plt.figure(figsize=(15, 3))
plt.plot(df_iot['date'], df_iot['Humidity'])
plt.show()
```



pie chart

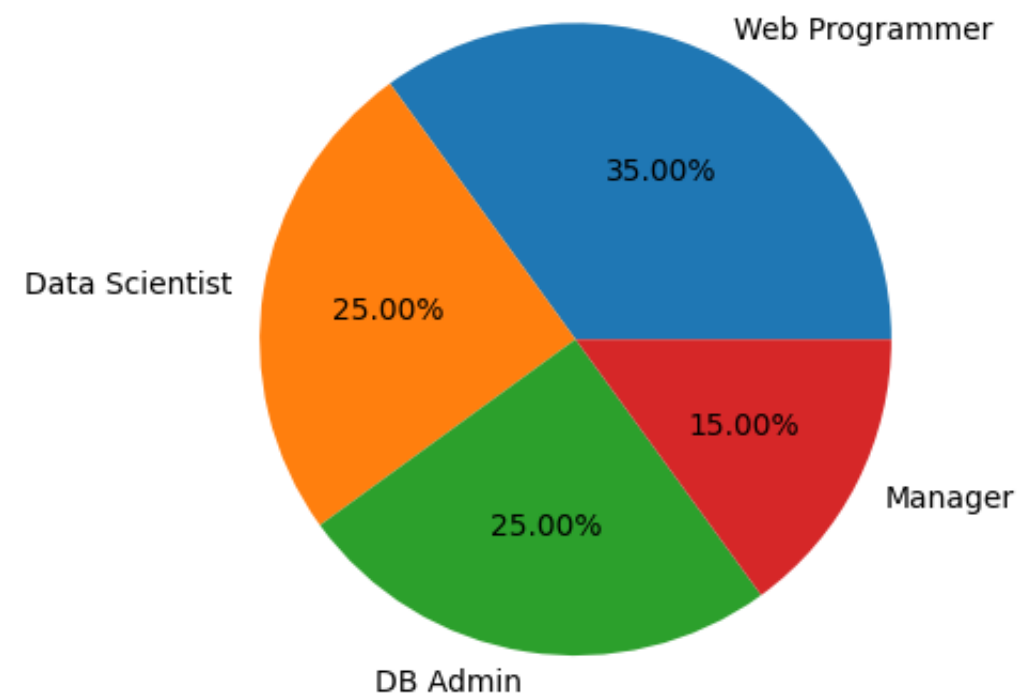
Pie chart adalah grafik lingkaran yang menunjukkan komposisi relatif dari bagian-bagian yang menyumbang pada keseluruhan, dengan setiap "slice" dari lingkaran mewakili proporsi atau persentase dari keseluruhan.

```
In [8]: # contoh dengan data manual/ data dummy
import matplotlib.pyplot as plt
import numpy as np

#https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.pie.html
y = np.array([35, 25, 25, 15])
mylabels = ["Web Programmer", "Data Scientist", "DB Admin", "Manager"]

plt.pie(y, labels = mylabels, autopct='%.2f%%')
plt.show()

# silahkan modifikasi data dari pie chart diatas, menggunakan contoh anda.
```

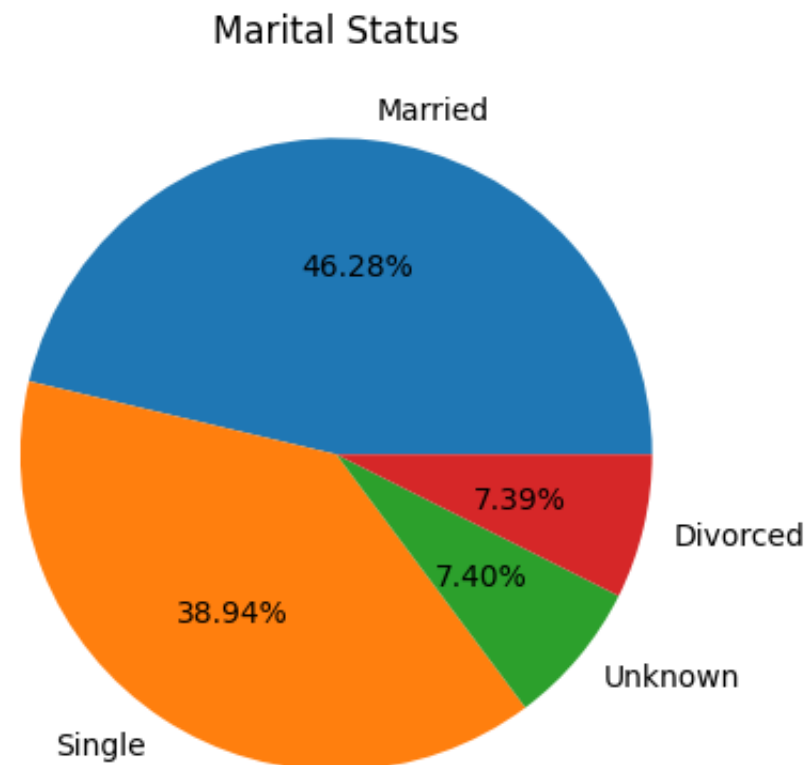


```
In [9]: # menggunakan dataset diatas
# versi 1. contoh menggunakan fungsi pie dari plt
import matplotlib.pyplot as plt

# fungsi value_counts() untuk melihat jumlah record unik dalam kolom
data = df_churning['Marital_Status'].value_counts()

# fungsi index untuk melihat label dari series
# https://pandas.pydata.org/docs/reference/api/pandas.Series.index.html
label = data.index

plt.pie(data, labels=label, autopct='%.2f%%')
#plt.legend(loc='upper left')
plt.title('Marital Status')
plt.show()
```

```
In [10]: # penjelasan
data = df_churning['Marital_Status'].value_counts()
print('keseluruhan data')
print(data)
print('index nya')
print(data.index)

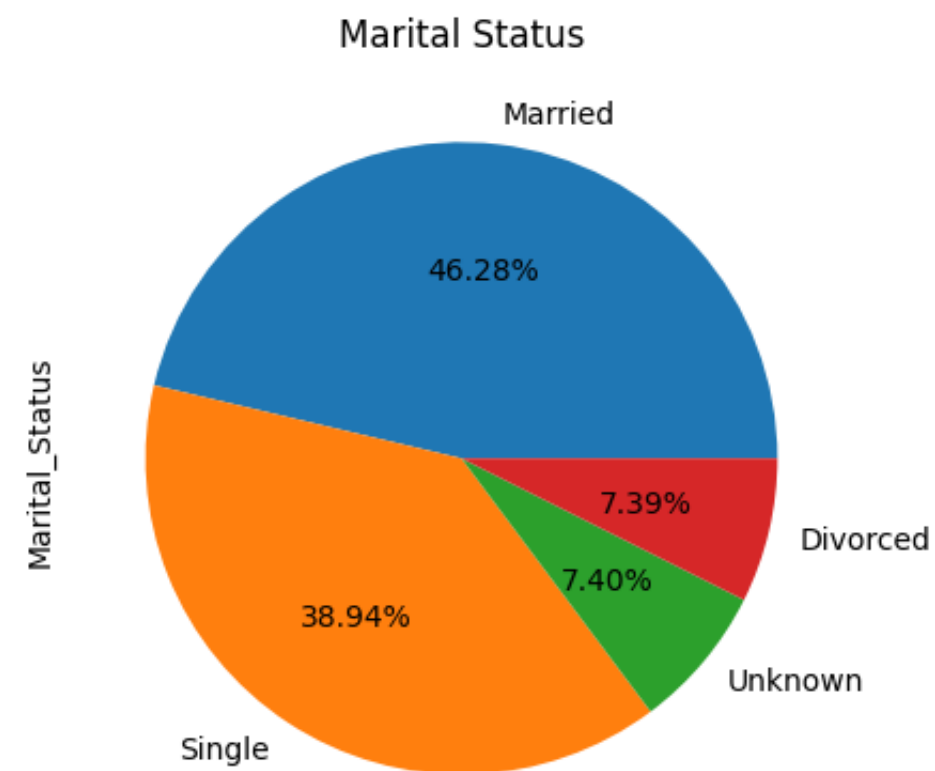
keseluruhan data
Married      4687
Single       3943
Unknown        749
Divorced       748
Name: Marital_Status, dtype: int64
index nya
Index(['Married', 'Single', 'Unknown', 'Divorced'], dtype='object')
```

```
In [11]: # menggunakan dataset diatas
# versi 2. contoh menggunakan fungsi plot.pie dari series secara langsung

data = df_churning['Marital_Status'].value_counts()
data.plot(kind='pie', autopct='%0.2f%%') # bisa pakai cara 1

#data.plot.pie(autopct='%0.2f%%') # bisa pakai cara 2

plt.title('Marital Status')
plt.show()
```



bar plot

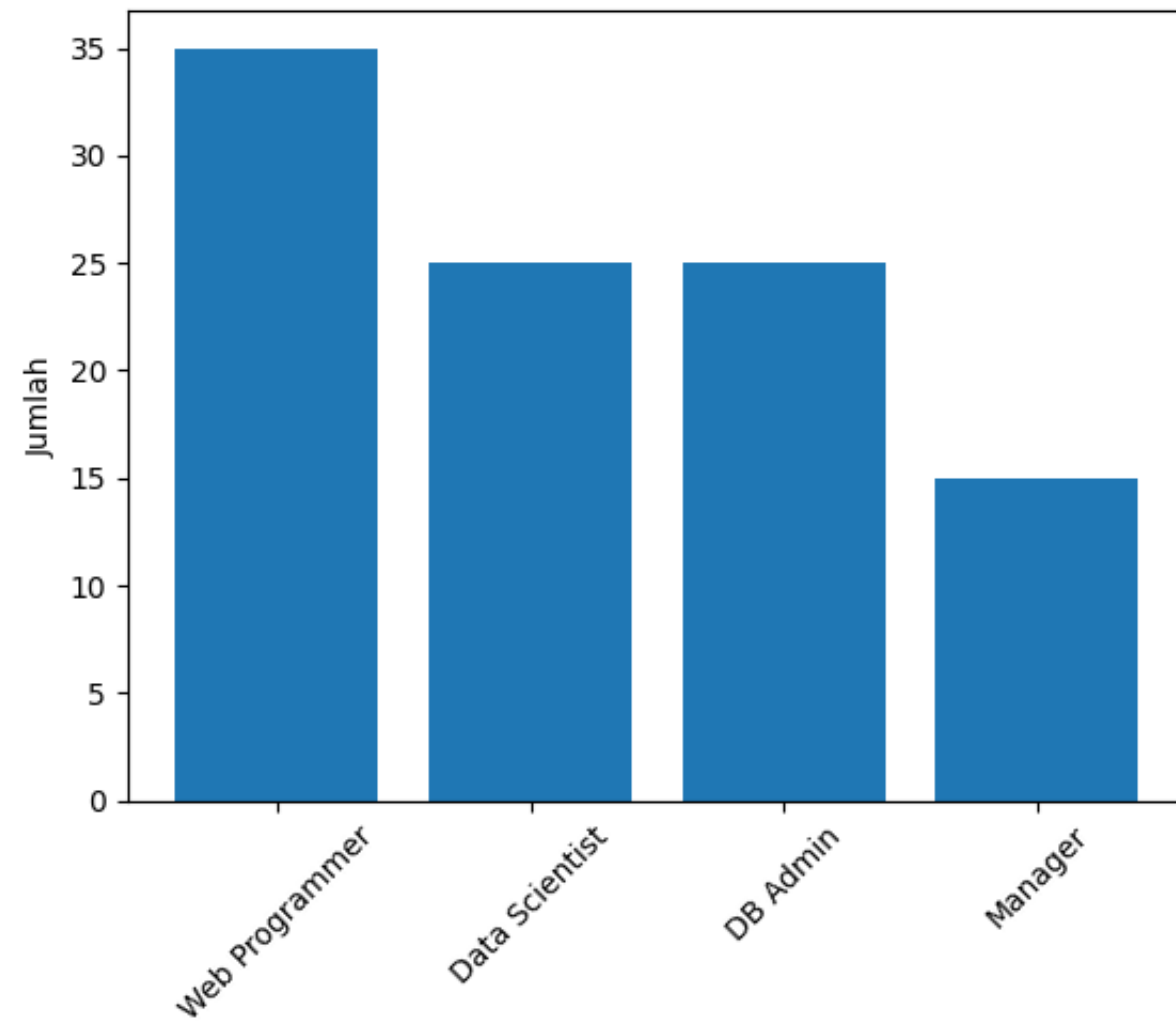
Bar plot adalah jenis grafik yang menampilkan nilai-nilai kategori dalam satu variabel dengan menggunakan batang vertikal atau horizontal untuk mewakili nilai atau frekuensi setiap kategori, memudahkan perbandingan antara kategori-kategori tersebut.

```
In [12]: #contoh dengan data dummy
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
x = ["Web Programmer", "Data Scientist", "DB Admin", "Manager"]

plt.bar(x,y)

plt.xticks(rotation=45)
plt.ylabel('Jumlah')
plt.show()
# silahkan modifikasi data dari bar plot diatas, menggunakan contoh anda.
```

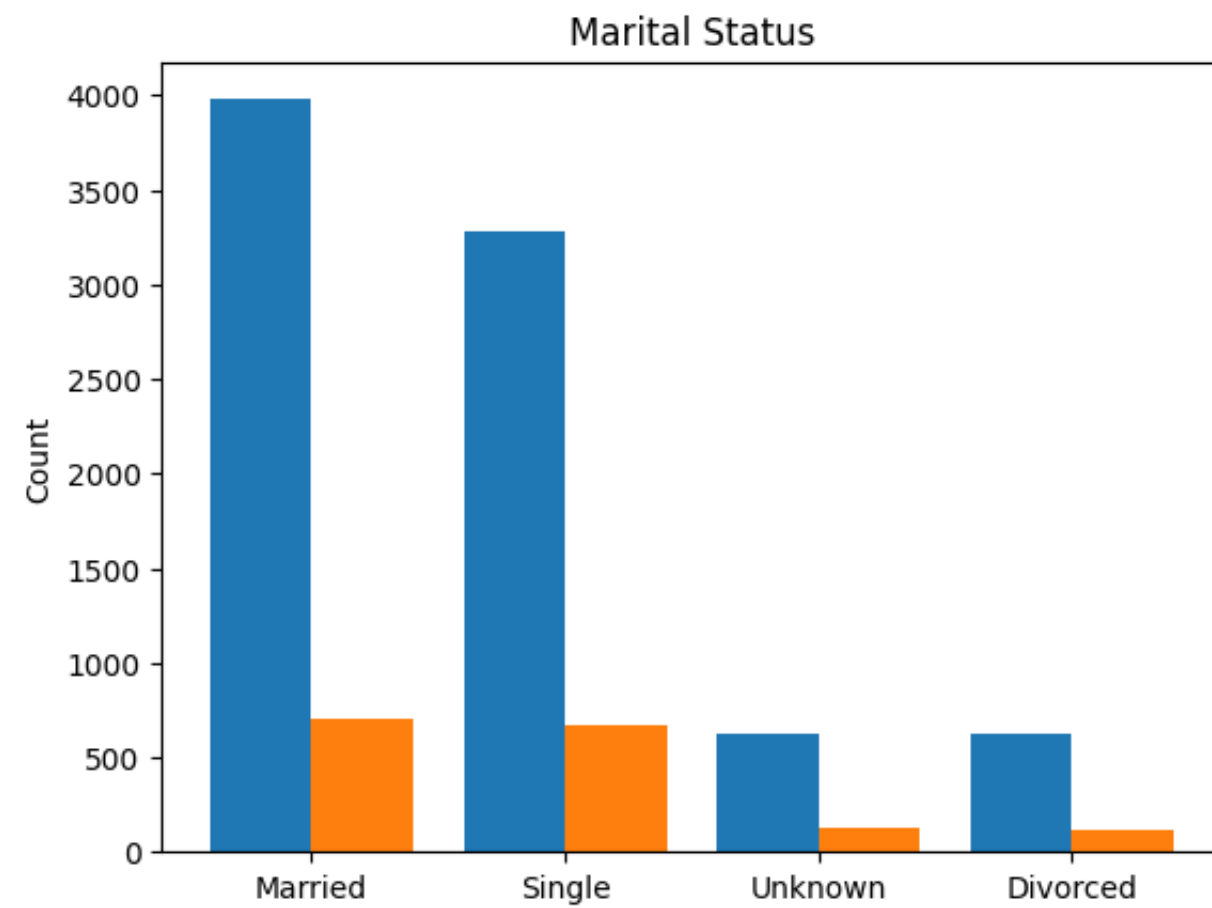
```
In [13]: # menggunakan dataset diatas
# versi 1 (agak susah). contoh menggunakan fungsi bar dari plt
y1 = existing_data['Marital_Status'].value_counts()
y2 = attrited_data['Marital_Status'].value_counts()

x1 = np.arange(len(y1.index))
x2 = np.arange(len(y2.index))
bar_width = 0.4

plt.bar(x1, y1, width=bar_width)
plt.bar(x2+bar_width, y2, width=bar_width)

plt.title('Marital Status')
plt.ylabel('Count')
plt.xticks(x2 + bar_width / 2, labels=df_churning.Marital_Status.unique())

plt.show()
```



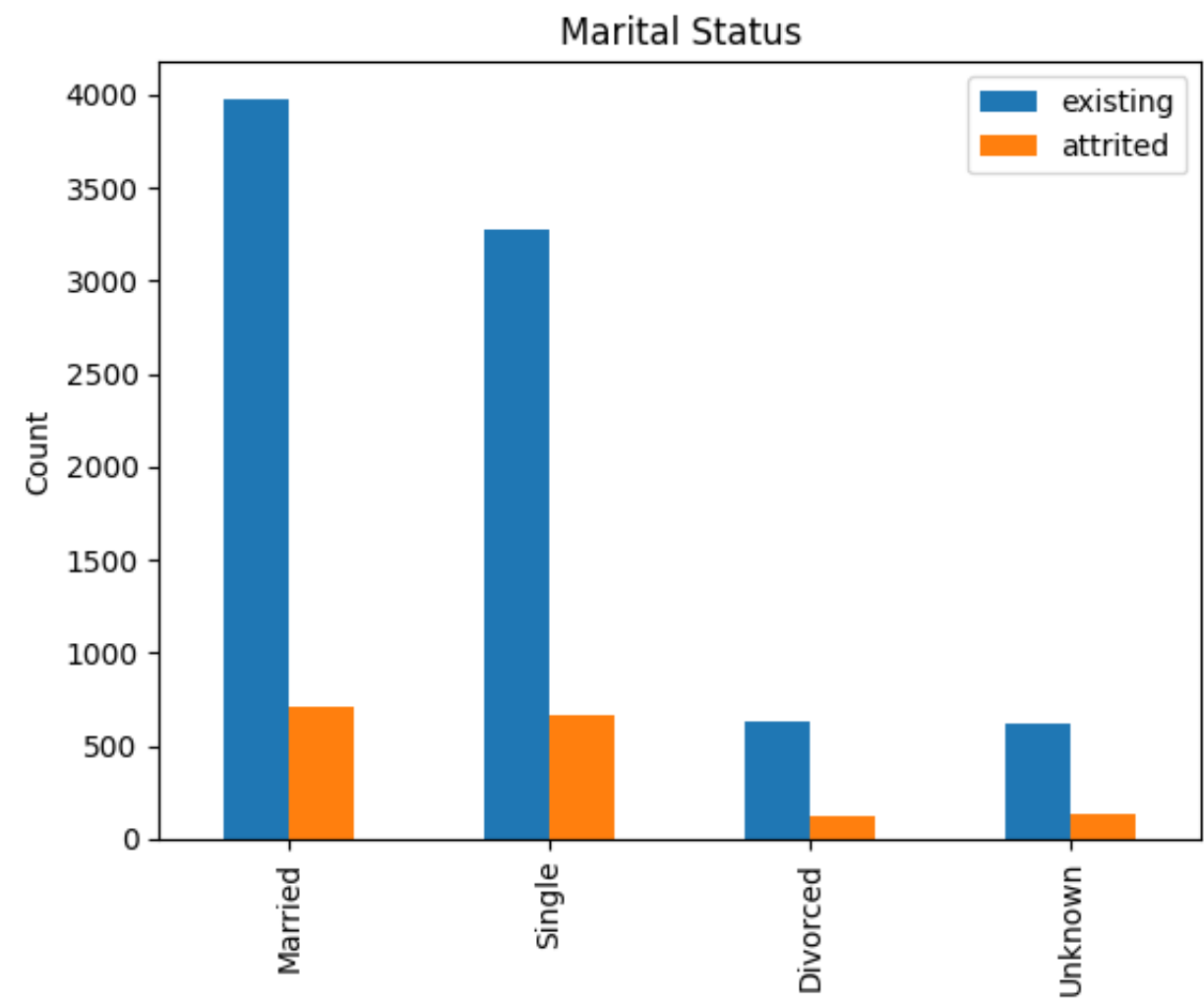
```
In [14]: # menggunakan dataset diatas
# versi 2 (lebih mudah). contoh menggunakan fungsi plot.bar dari object dataframe
data1 = existing_data['Marital_Status'].value_counts()
data2 = attrited_data['Marital_Status'].value_counts()

df_new = pd.concat([data1, data2], keys=['existing', 'attrited'], axis=1)

#df_new.plot.bar() #bisa pakai cara 1
df_new.plot(kind='bar') # bisa pakai cara 2

plt.title('Marital Status')
plt.ylabel('Count')

plt.show()
```



```
In [15]: # lihat dataframe baru yang berhasil dibuat, penggabungan dari 2 series
df_new
```

Out[15]:

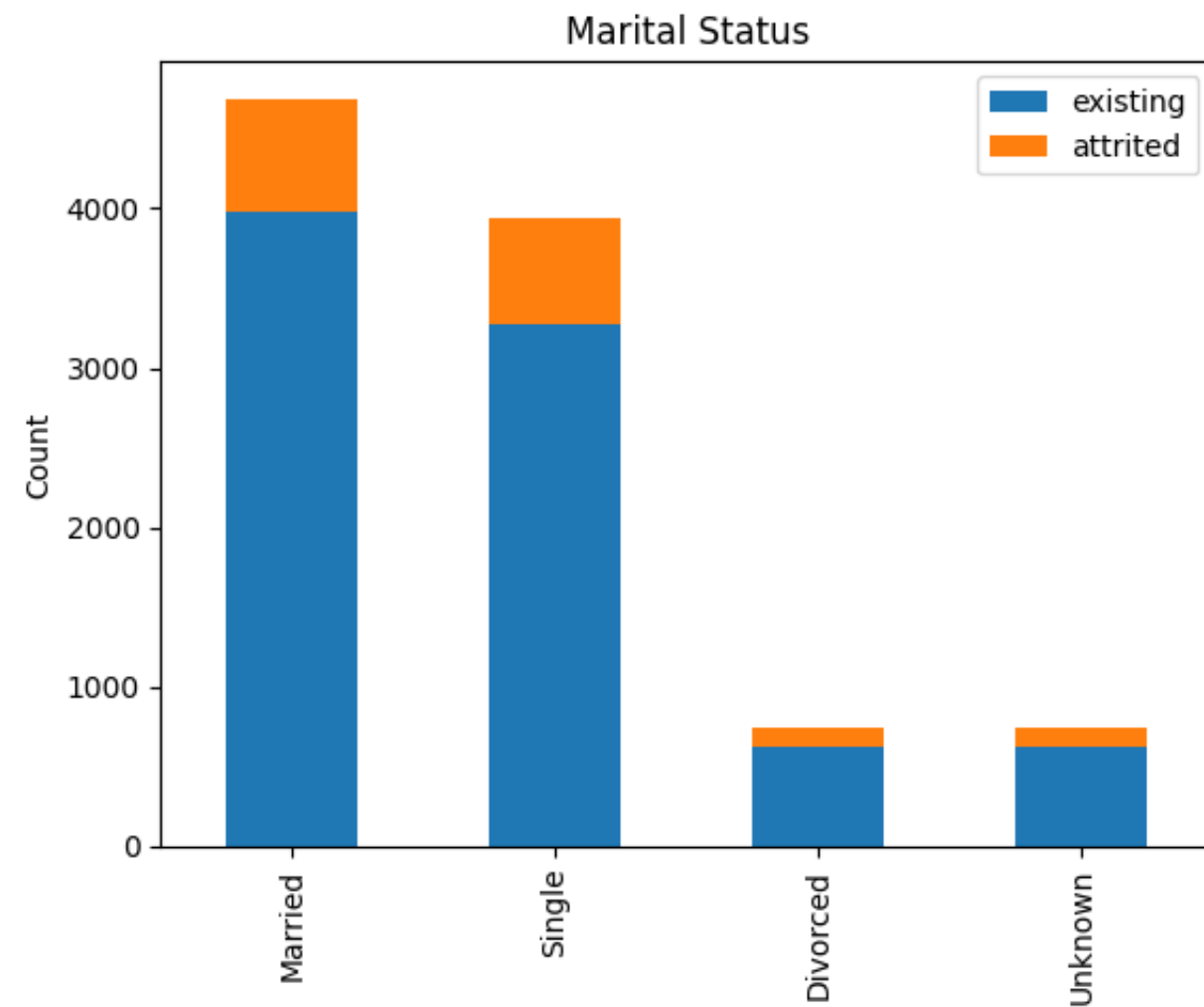
	existing	attrited
Married	3978	709
Single	3275	668
Divorced	627	121
Unknown	620	129

stacked bar plot

Stacked bar plot adalah jenis grafik yang menunjukkan komposisi variabel dalam satu kategori dengan menggunakan batang vertikal atau horizontal yang ditumpuk satu sama lain untuk mewakili kontribusi relatif dari setiap variabel pada kategori tersebut, sehingga memungkinkan untuk melihat hubungan antara variabel-variabel tersebut dalam satu kategori.

```
In [16]: # menggunakan dataset diatas
# contoh menggunakan stacked bar plot
data1 = existing_data['Marital_Status'].value_counts()
data2 = attrited_data['Marital_Status'].value_counts()
df_new = pd.concat([data1, data2], keys=['existing', 'attrited'], axis=1)

#df_new.plot.bar() #bisa pakai cara 1
df_new.plot(kind='bar', stacked=True) # bisa pakai cara 2
plt.title('Marital Status')
plt.ylabel('Count')
plt.show()
```



```
In [17]: # contoh membuat dataframe baru, yang menjelaskan value dan persentase
df_new['existing_percent'] = (df_new['existing'] /
                             (df_new['existing'] + df_new['attrited'])) * 100
df_new['attrited_percent'] = (df_new['attrited'] /
                              (df_new['attrited'] + df_new['existing'])) * 100
df_new
```

Out[17]:

	existing	attrited	existing_percent	attrited_percent
Married	3978	709	84.873053	15.126947
Single	3275	668	83.058585	16.941415
Divorced	627	121	83.823529	16.176471
Unknown	620	129	82.777036	17.222964

histogram

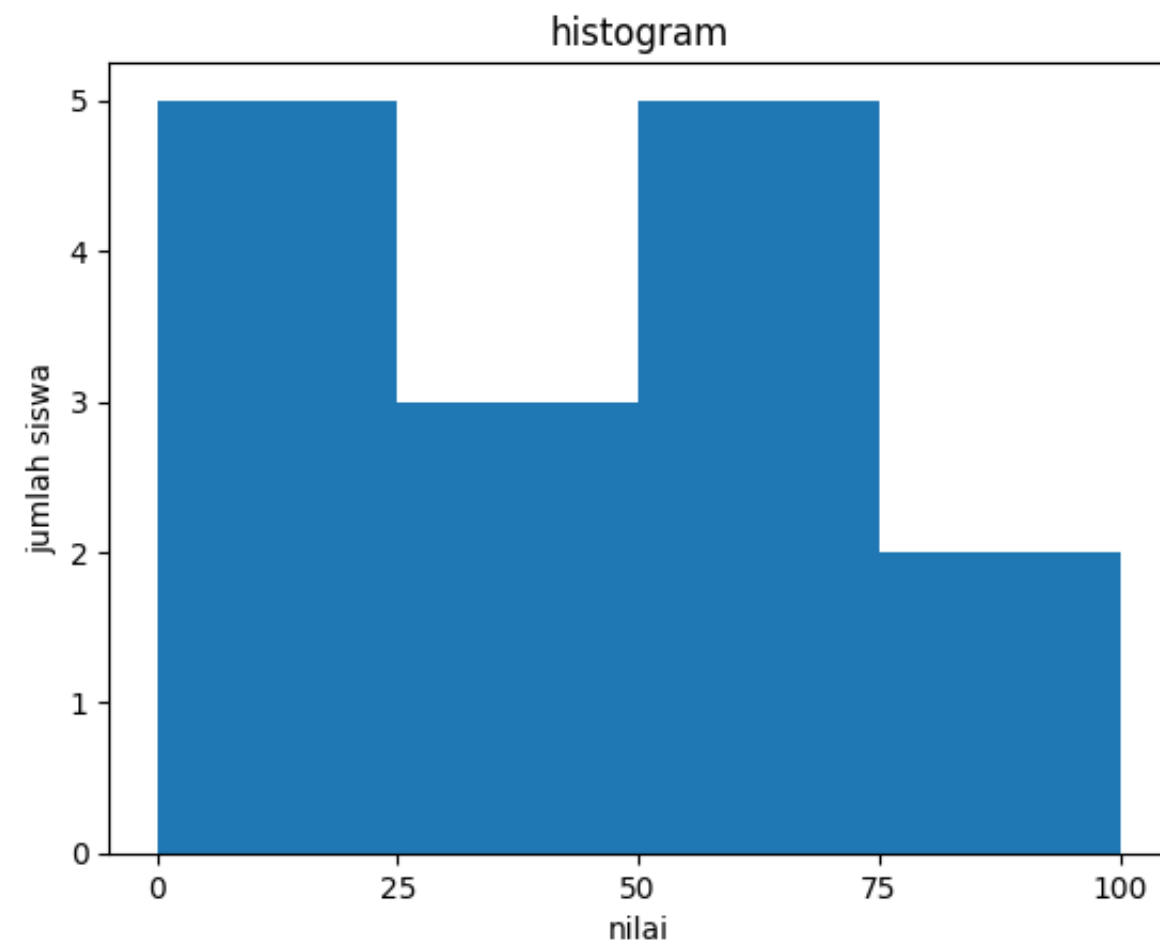
Histogram adalah grafik yang menampilkan distribusi frekuensi atau probabilitas dari data numerik dengan membagi rentang nilai data menjadi interval atau "bin" dan menunjukkan seberapa sering nilai-nilai tersebut muncul dalam setiap interval.

In [18]:

```
# dengan data manual
data = np.array([22,87,5,43,56,73,55,54,11,20,51,5,79,31,27])
bins = [0,25,50,75,100]

plt.hist(data, bins = bins)
plt.title("histogram")
plt.xticks([0,25,50,75,100])
plt.xlabel('nilai')
plt.ylabel('jumlah siswa')
plt.show()

# silahkan modifikasi menggunakan data anda sendiri
```

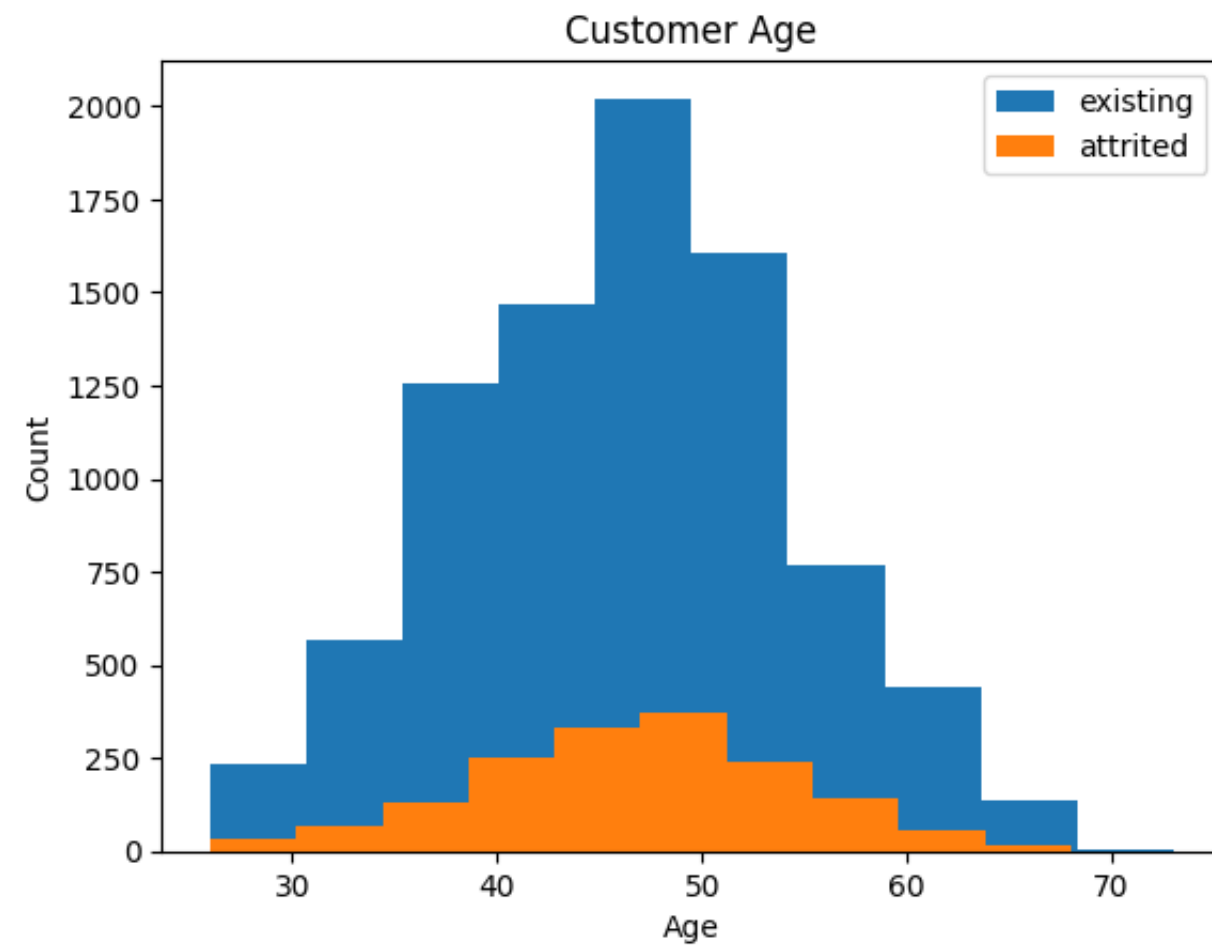


```
In [19]: # menggunakan dataset diatas
# versi 1. contoh menggunakan fungsi hist dari plt
data1 = existing_data['Customer_Age']
data2 = attrited_data['Customer_Age']

plt.hist(data1, label='existing')
plt.hist(data2, label='attrited')

plt.title('Customer Age')
plt.ylabel('Count')
plt.xlabel('Age')

plt.legend()
plt.show()
```

Tugas 1 (lanjutan)

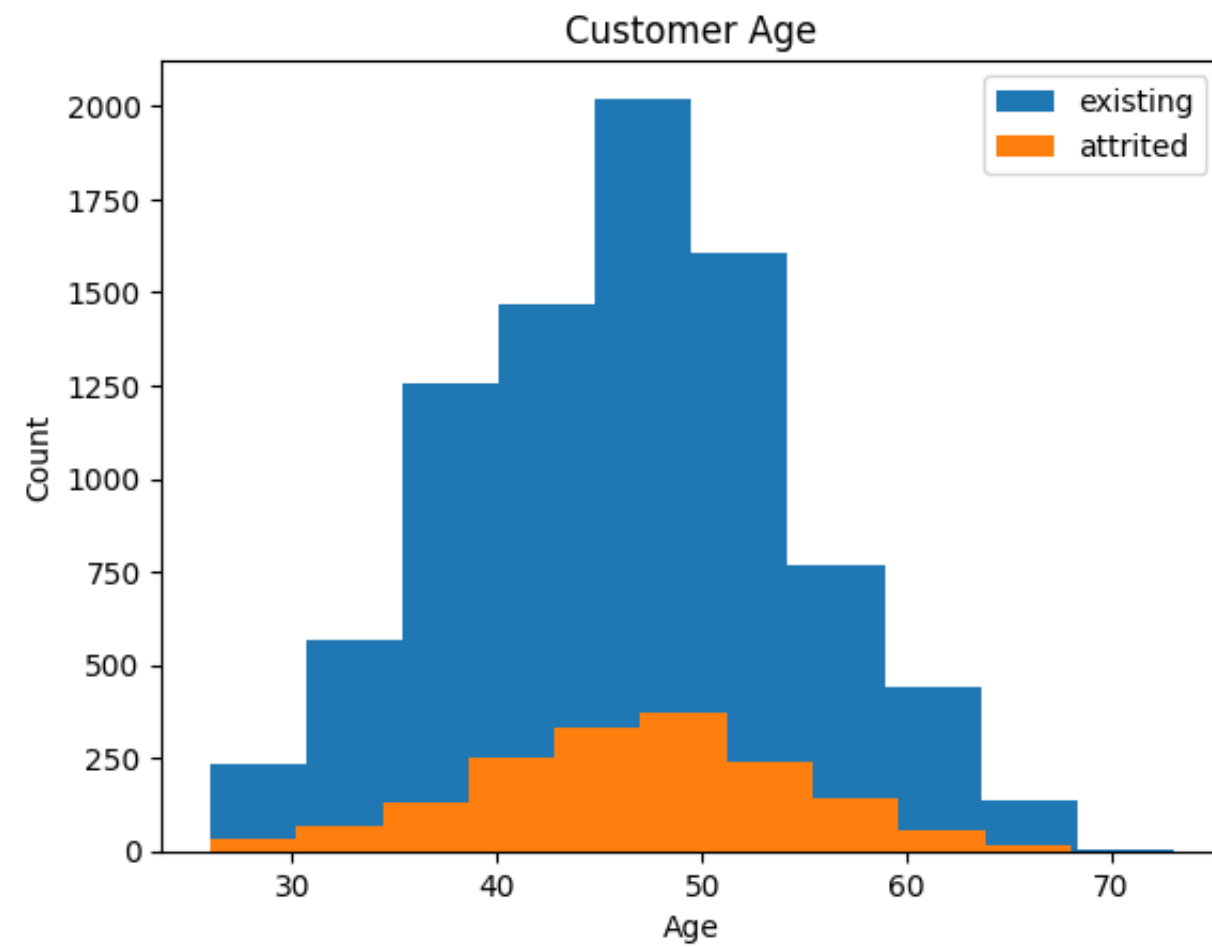
6. Dari gambar histogram diatas, buatlah penjelasannya nya!

```
In [20]: # menggunakan dataset diatas
# versi 2. contoh menggunakan fungsi plot.hist dari object dataframe
import matplotlib.pyplot as plt

existing_data['Customer_Age'].plot.hist(label='existing')
attrited_data['Customer_Age'].plot.hist(label='attrited')
#attrited_data['Customer_Age'].plot(kind='hist') #bisa juga pakai cara ini

plt.title('Customer Age')
plt.ylabel('Count')
plt.xlabel('Age')

plt.legend()
plt.show()
```



scatter plot

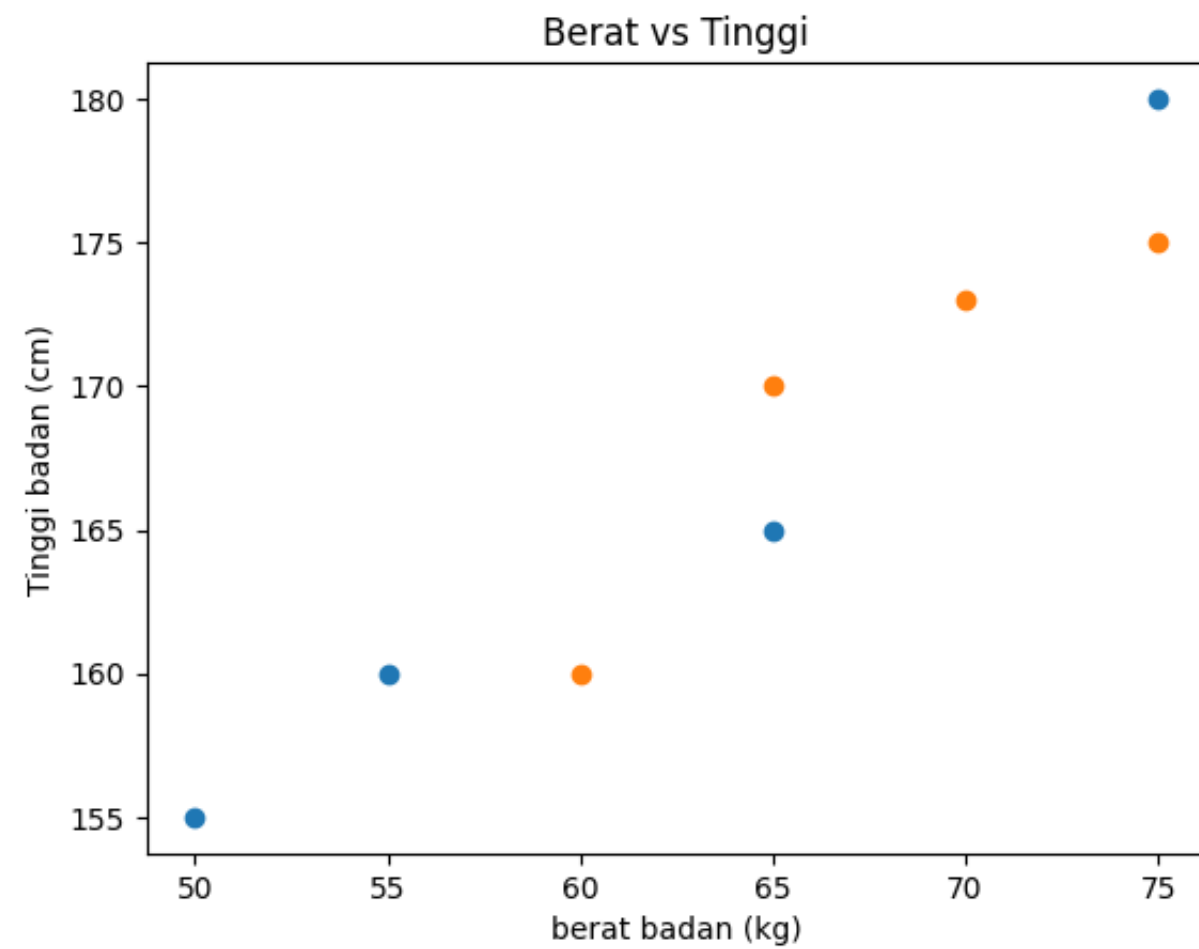
Scatter plot adalah jenis grafik yang menampilkan titik-titik data dalam bidang kartesian, di mana setiap titik mewakili pengamatan individu dan posisinya ditentukan oleh nilai dari dua variabel, memungkinkan untuk mengeksplorasi hubungan atau pola antara kedua variabel tersebut.

```
In [21]: #data manual
berat_a = np.array([50, 65, 75, 55])
tinggi_a = np.array([155, 165, 180, 160])

berat_b = np.array([60, 75, 65, 70])
tinggi_b = np.array([160, 175, 170, 173])

plt.scatter(berat_a, tinggi_a)
plt.scatter(berat_b, tinggi_b)

plt.title('Berat vs Tinggi')
plt.ylabel('Tinggi badan (cm)')
plt.xlabel('berat badan (kg)')
plt.show()
```

```
In [22]: # menggunakan dataset
# versi 1. contoh menggunakan fungsi scatter dari plt
y1 = attrited_data['Months_on_book']
x1 = attrited_data['Customer_Age']

y2 = existing_data['Months_on_book']
x2 = existing_data['Customer_Age']

plt.scatter(x2, y2)
plt.scatter(x1, y1)

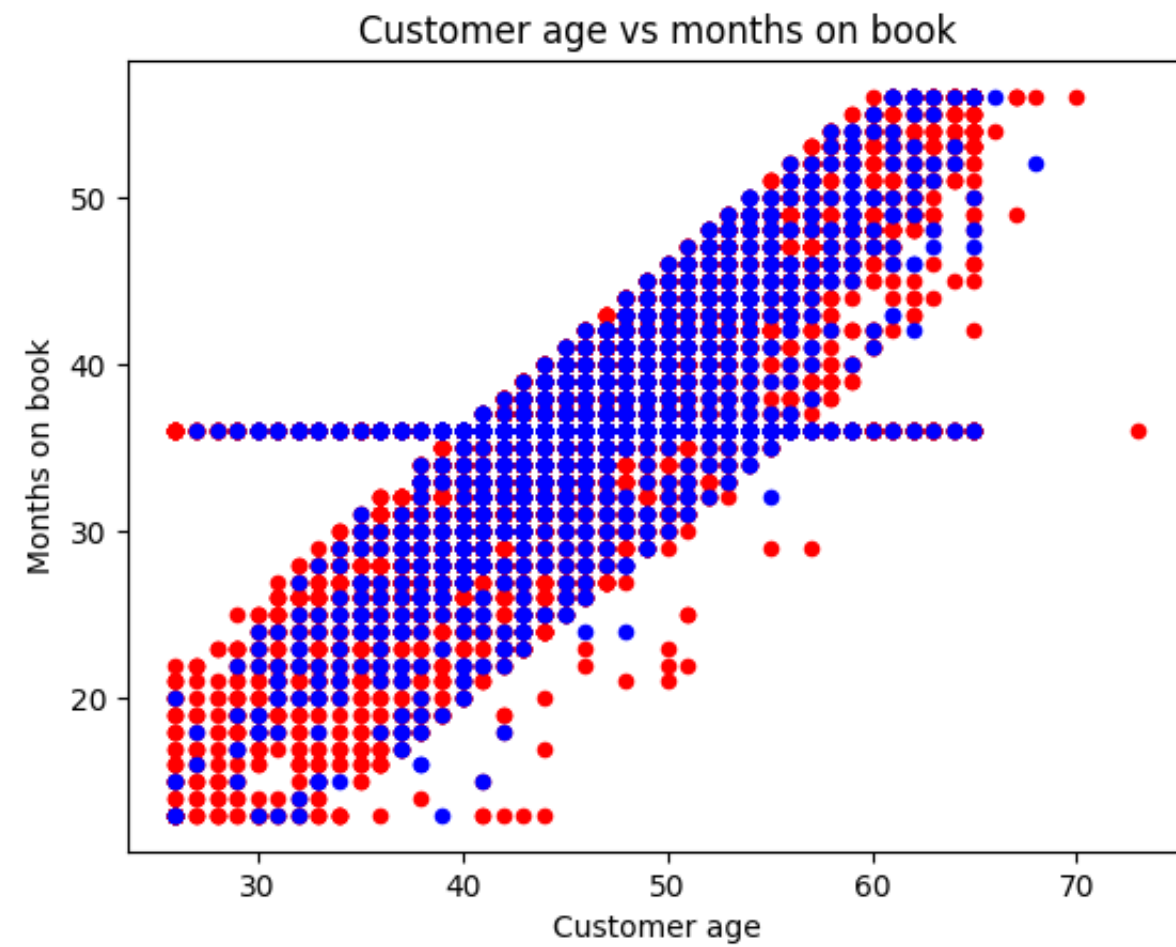
plt.title('Customer age vs months on book')
plt.ylabel('Months on book')
plt.xlabel('Customer age')

plt.show()
```



```
In [23]: # menggunakan dataset diatas
# versi 2. contoh menggunakan fungsi plot.scatter dari object dataframe
ax=existing_data.plot.scatter(x='Customer_Age', y='Months_on_book', c='red')
attrited_data.plot.scatter(x='Customer_Age', y='Months_on_book', c='blue', ax=ax)

plt.title('Customer age vs months on book')
plt.ylabel('Months on book')
plt.xlabel('Customer age')
plt.show()
```

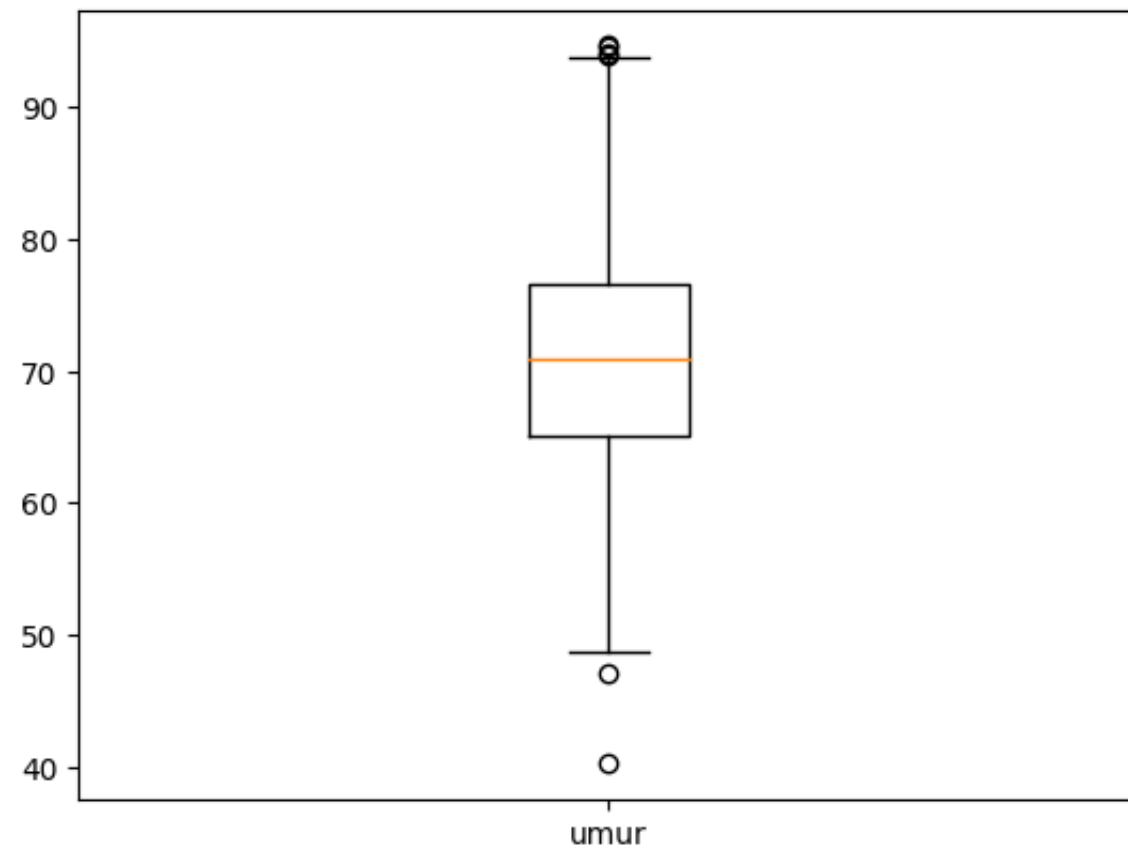


boxplot

Box plot adalah jenis grafik statistik yang menampilkan distribusi data numerik melalui kotak (box) yang menunjukkan kuartil data, garis yang menunjukkan median, dan garis-garis yang menunjukkan rentang interkuartil, serta titik-titik yang menunjukkan outlier potensial, memungkinkan untuk melihat ringkasan visual tentang distribusi data dan mengidentifikasi outlier.

```
In [24]: #contoh data manual
np.random.seed(10)
data = np.random.normal(70, 10, 200) #buat data random, rata2 70, std dev 10, jumlah data 200

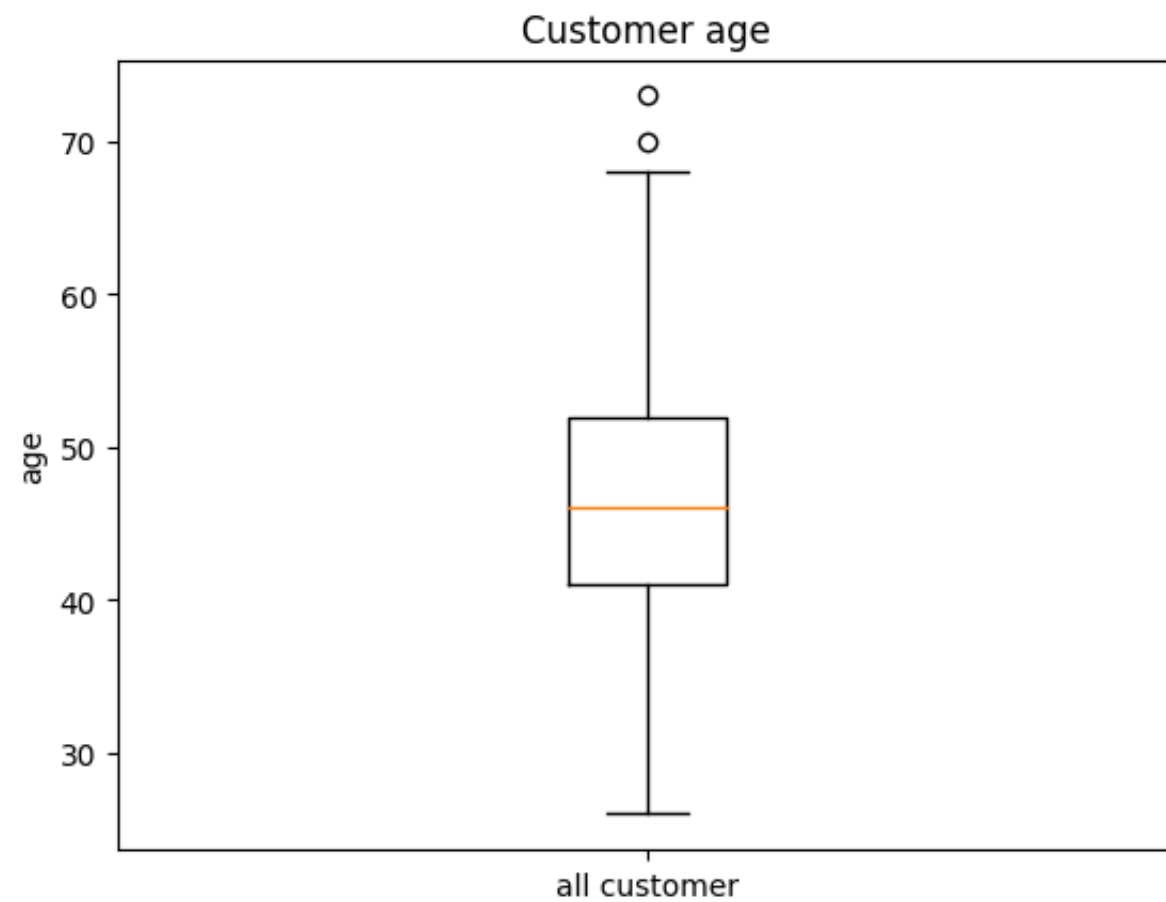
plt.boxplot(data)
plt.xticks([1], labels=['umur'])
plt.show()
```



```
In [25]: # menggunakan dataset diatas
# versi 1. contoh menggunakan fungsi boxplot dari plt. single group.
plt.boxplot(df_churning['Customer_Age'])

plt.title('Customer age')
plt.ylabel('age')
plt.xticks([1], labels=['all customer'])
plt.show()

#
```



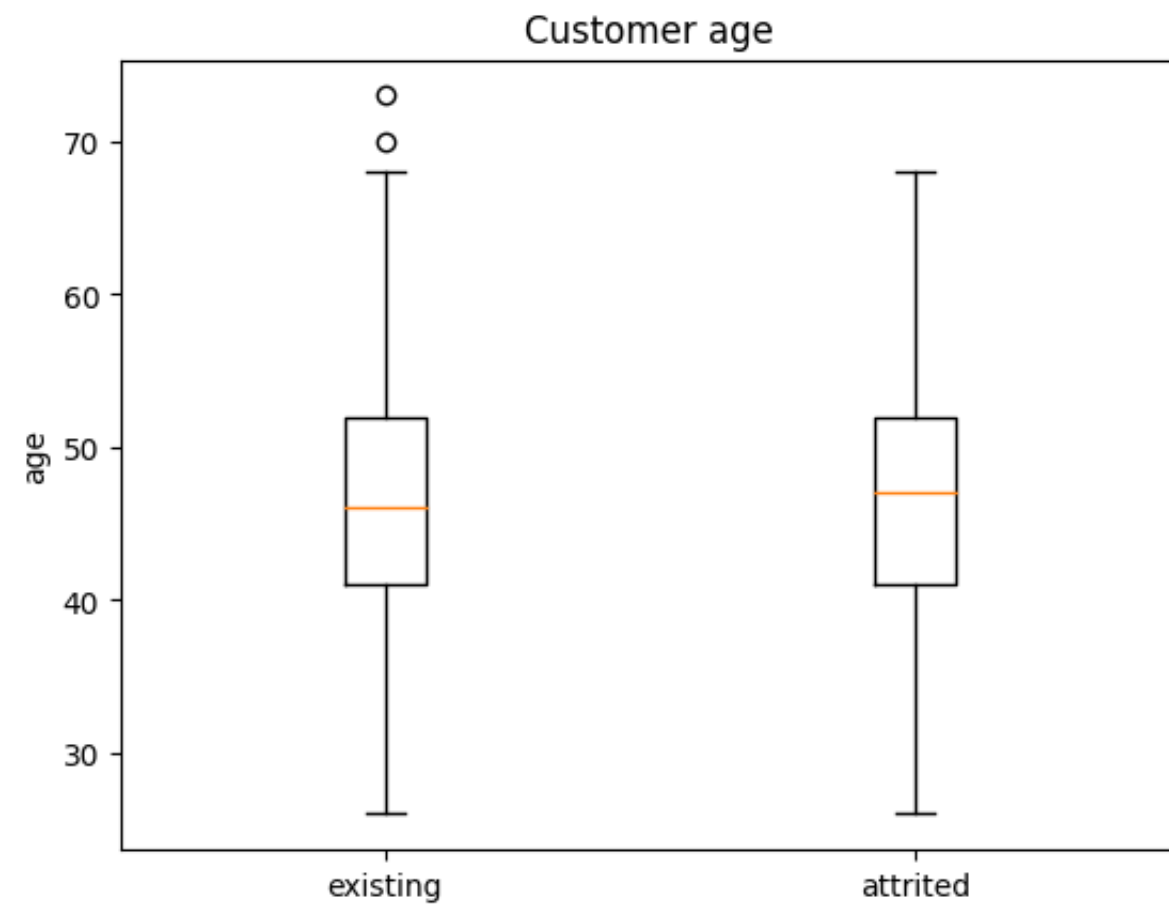
Tugas 1 (lanjutan)

7. jelaskan gambar box plot diatas, apakah ada data outlier?

```
In [26]: # menggunakan dataset diatas
# versi 1. contoh menggunakan fungsi boxplot dari plt. multiple group.

data1 = existing_data['Customer_Age'].values
data2 = attrited_data['Customer_Age'].values
#buat object dictionary dahulu
my_dictionary = {'existing': data1, 'attrited': data2}
#ambil value nya
plt.boxplot(list(my_dictionary.values()))

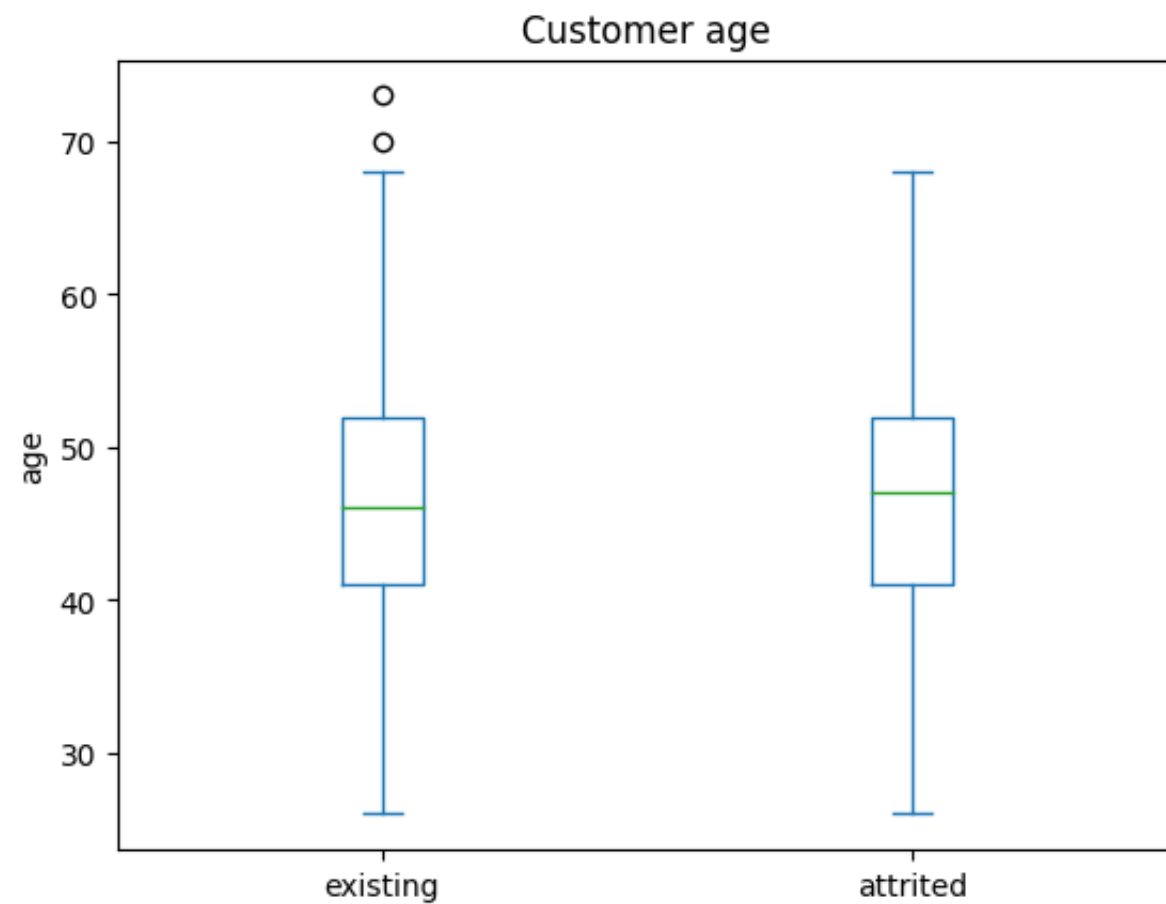
plt.title('Customer age')
plt.ylabel('age')
plt.xticks([1,2], labels=list(my_dictionary.keys()))
plt.show()
```



```
In [27]: # menggunakan dataset diatas
# versi 2. contoh menggunakan fungsi plot.box dari object dataframe
data1 = existing_data['Customer_Age']
data2 = attrited_data['Customer_Age']
df_new = pd.concat([data1, data2], keys=['existing', 'attrited'], axis=1)

df_new.plot.box() #bisa pakai cara ini
#df_new.plot(kind='box') #bisa menggunakan cara ini juga

plt.title('Customer age')
plt.ylabel('age')
plt.show()
```

Tugas 1 (lanjutan)

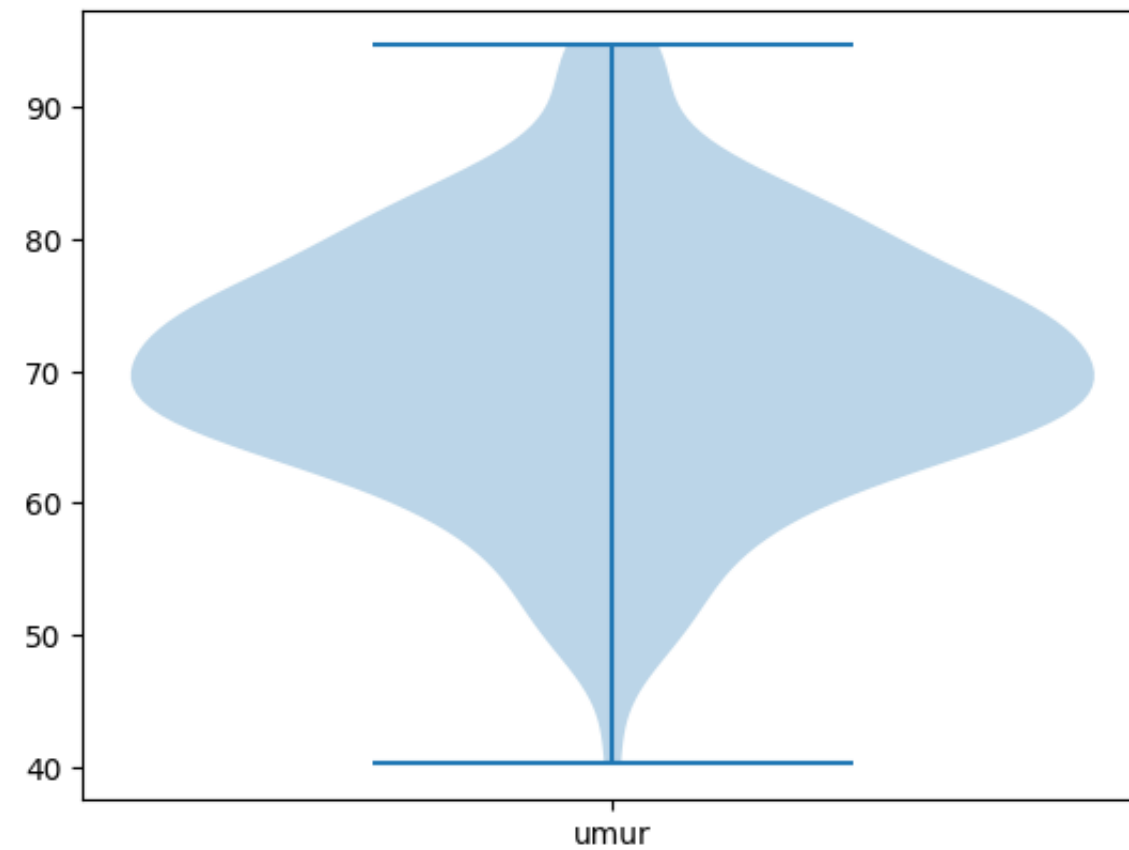
8. Jelaskan gambar box plot diatas. apa perbedaan customer existing dan attrited? apa itu customer existing dan attrited?

violin plot

Violin plot adalah jenis grafik yang menampilkan distribusi data numerik dengan menggunakan bentuk mirip biola yang menunjukkan kerapatan relatif dari nilai data di berbagai bagian rentang nilai, memungkinkan untuk melihat bentuk distribusi data secara lebih rinci daripada histogram dan box plot.

```
In [28]: #contoh data manual
np.random.seed(10)
data = np.random.normal(70, 10, 200) #buat data random, rata2 70, std dev 10, jumlah data 200
plt.violinplot(data)

plt.xticks([1], labels=['umur'])
plt.show()
```



In [29]: *# menggunakan dataset diatas*
versi 1. contoh menggunakan fungsi violinplot dari plt. multiple group.

```
data1 = existing_data['Customer_Age'].values
data2 = attrited_data['Customer_Age'].values
#buat object dictionary dahulu
my_dictionary = {'existing': data1, 'attrited': data2}
#ambil value nya
plt.violinplot(list(my_dictionary.values()))

plt.title('Customer age')
plt.ylabel('age')
plt.xticks([1,2], labels=list(my_dictionary.keys()))
plt.show()
```




Tambahan

Jika waktu praktikum tidak cukup, bagian ini bisa dicoba dirumah. Silahkan lanjut ke datapreprocessing

Sub plot

Subplot adalah fitur dalam visualisasi data yang memungkinkan pengguna untuk membuat beberapa grafik atau plot secara bersamaan dalam satu gambar atau "figure", memudahkan perbandingan atau representasi visual dari beberapa dataset atau hasil analisis secara paralel.

```
In [30]: #versi 1
import matplotlib.pyplot as plt
import numpy as np

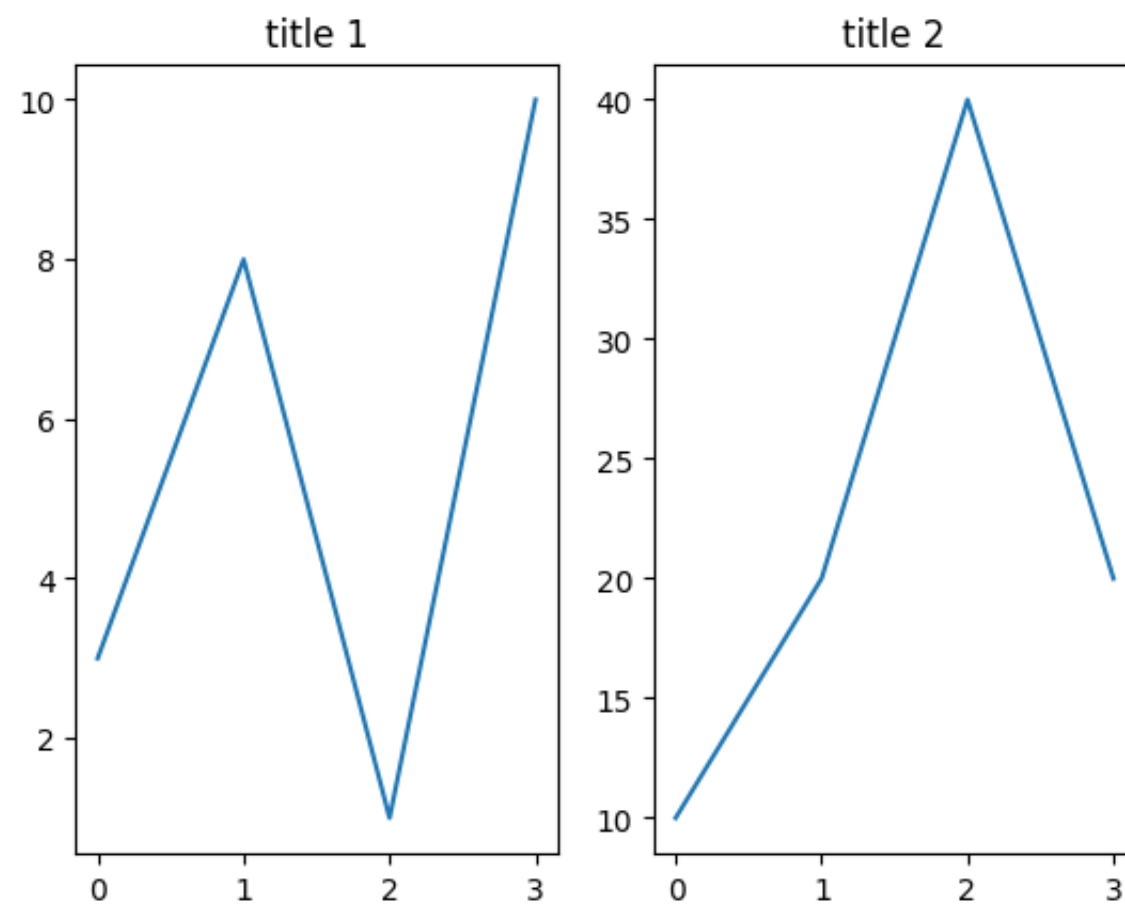
#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(1, 2, 1) #1 baris, 2 kolom, dan plot ini adalah plot pertama.
plt.plot(x,y)
plt.title('title 1')

#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 40, 20])

plt.subplot(1, 2, 2) #1 baris, 2 kolom, dan plot ini adalah plot kedua.
plt.plot(x,y)
plt.title('title 2')

plt.show()
```



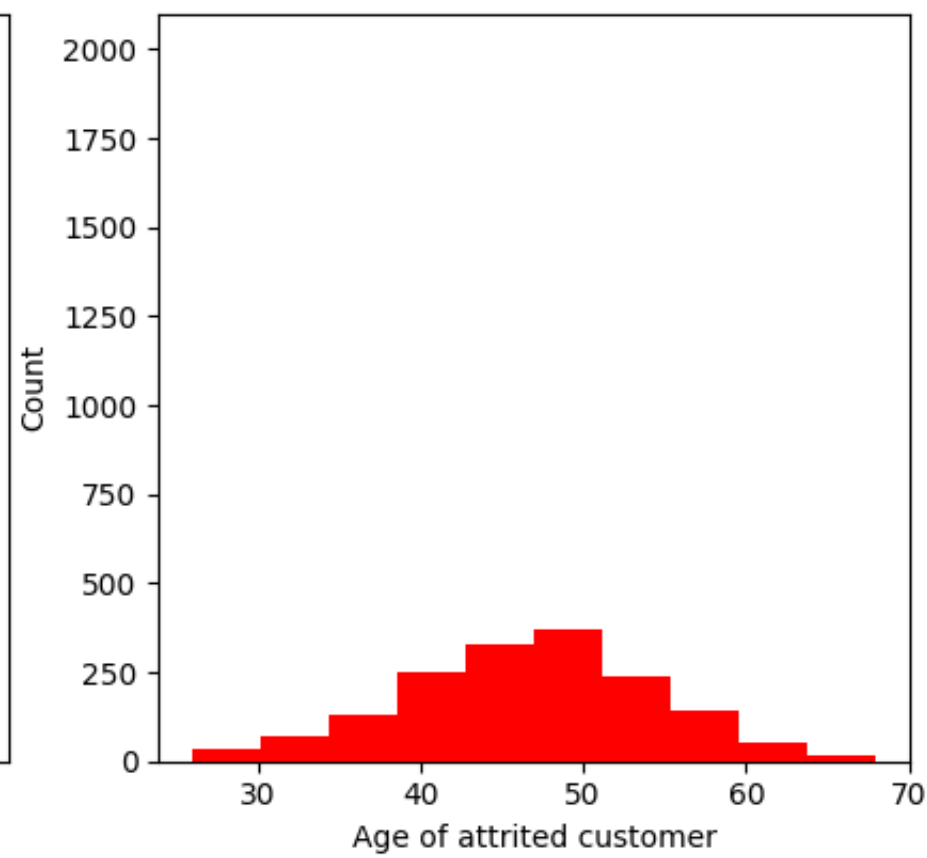
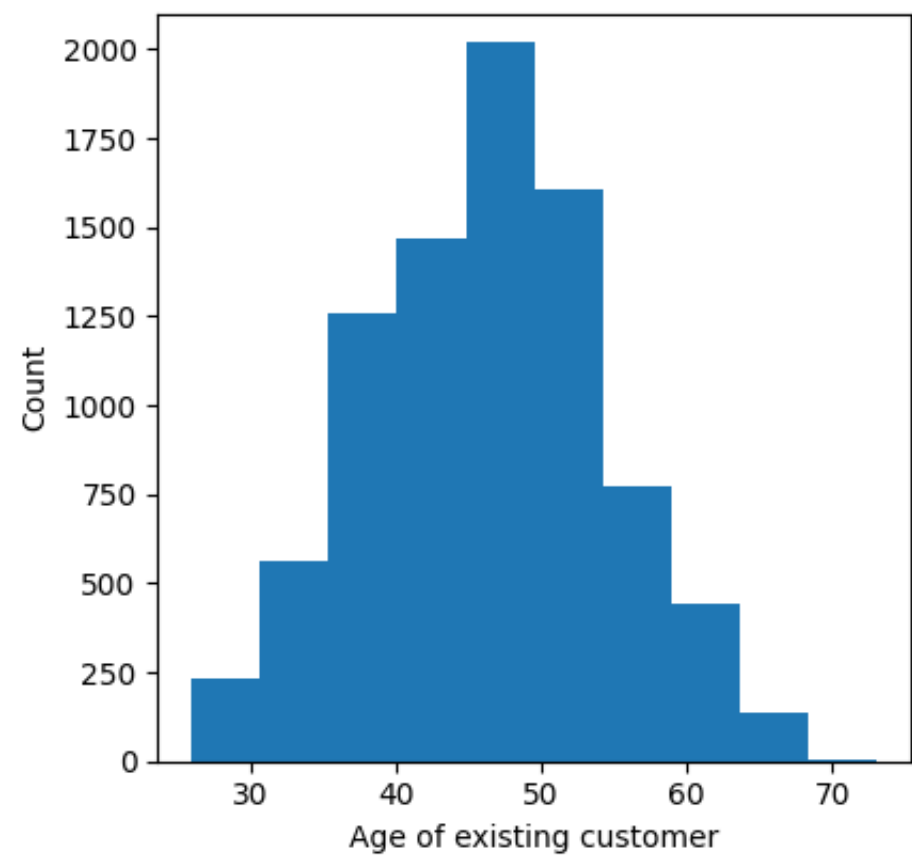
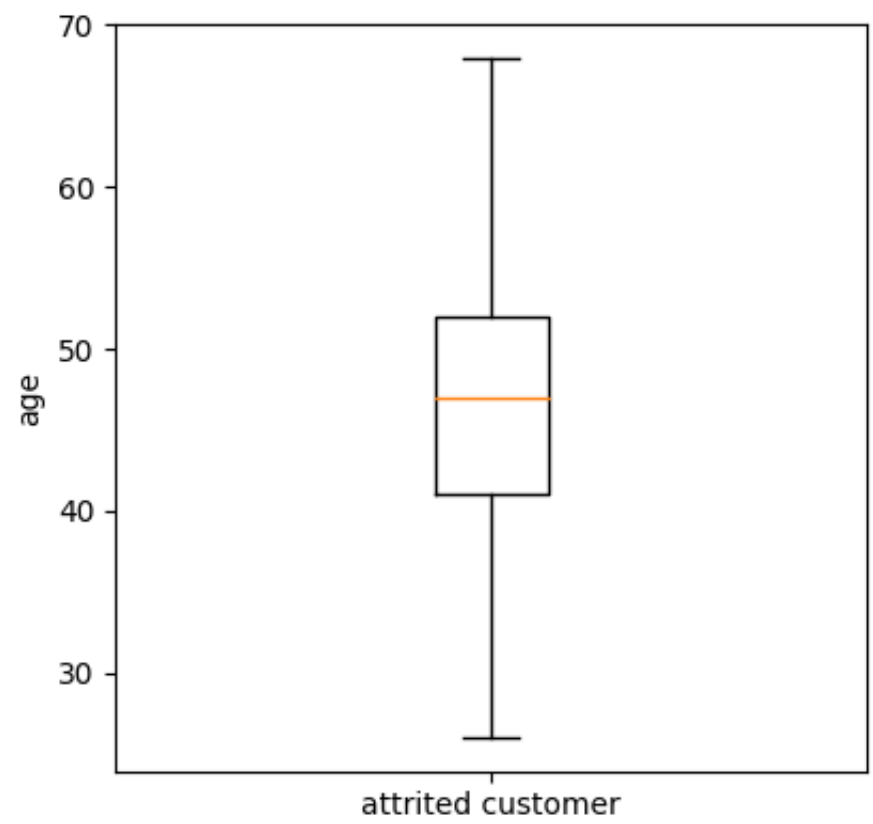
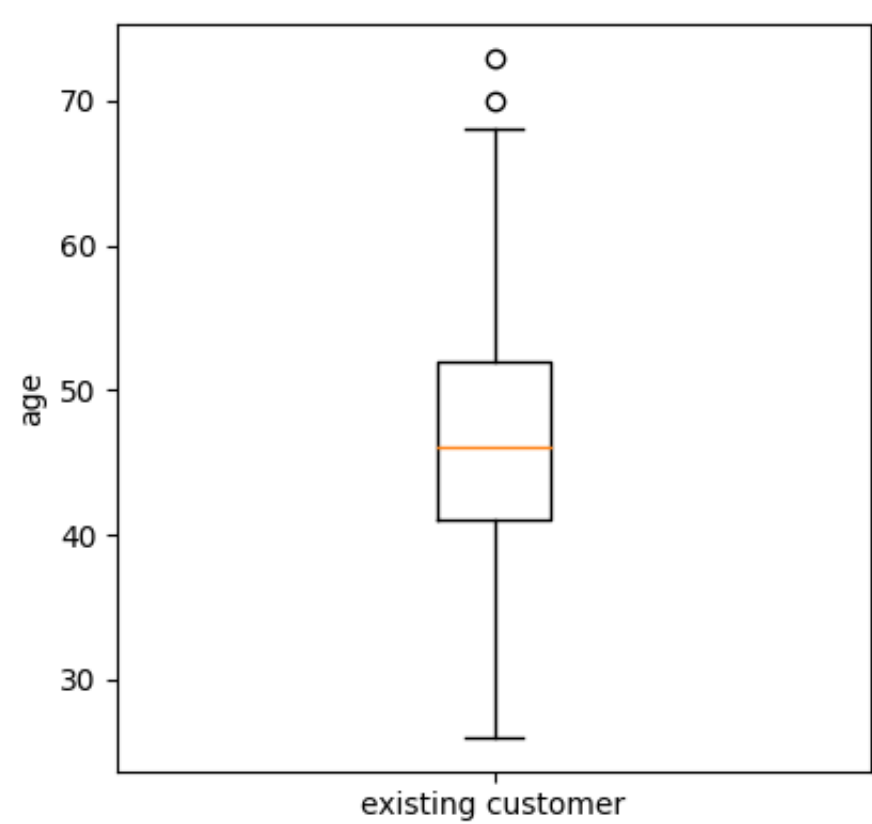
```
In [31]: #versi 1
plt.figure(figsize=(10, 10))
plt.subplot(2, 2, 1) #2 baris, 2 kolom, plot 1
plt.boxplot(existing_data['Customer_Age'])
plt.ylabel('age')
plt.xticks([1], labels=['existing customer'])

plt.subplot(2, 2, 2)
plt.boxplot(attrited_data['Customer_Age'])
plt.ylabel('age')
plt.xticks([1], labels=['attrited customer'])

plt.subplot(2, 2, 3)
existing_data['Customer_Age'].plot.hist()
plt.ylabel('Count')
plt.xlabel('Age of existing customer')
plt.ylim([0, 2100])

plt.subplot(2, 2, 4)
attrited_data['Customer_Age'].plot.hist(color='r')
plt.ylabel('Count')
plt.xlabel('Age of attrited customer')
plt.ylim([0, 2100])

plt.show()
```



```
In [32]: #versi 2
fig, ax = plt.subplots(2,2, figsize=(10, 10))

ax[0,0].boxplot(existing_data['Customer_Age'])
ax[0,0].set_ylabel('age')
ax[0,0].set_xticklabels(['existing customer'])

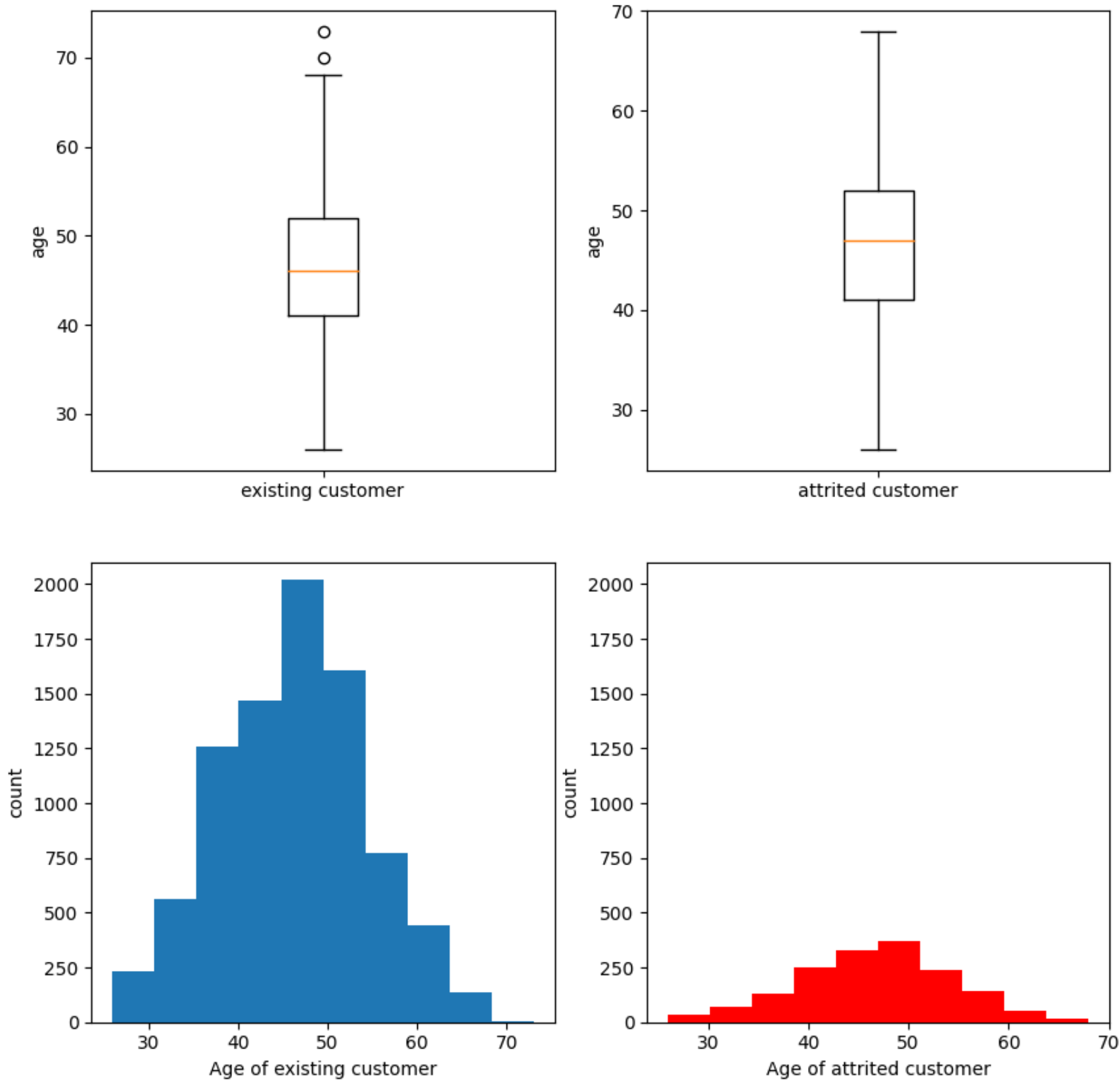
ax[0,1].boxplot(attrited_data['Customer_Age'])
ax[0,1].set_ylabel('age')
ax[0,1].set_xticklabels(['attrited customer'])

data1 = existing_data['Customer_Age']
data2 = attrited_data['Customer_Age']

ax[1,0].hist(data1)
ax[1,0].set_xlabel('Age of existing customer')
ax[1,0].set_ylabel('count')
ax[1,0].set_ylim([0, 2100])

ax[1,1].hist(data2, color='r')
ax[1,1].set_xlabel('Age of attrited customer')
ax[1,1].set_ylabel('count')
ax[1,1].set_ylim([0, 2100])

plt.show()
```

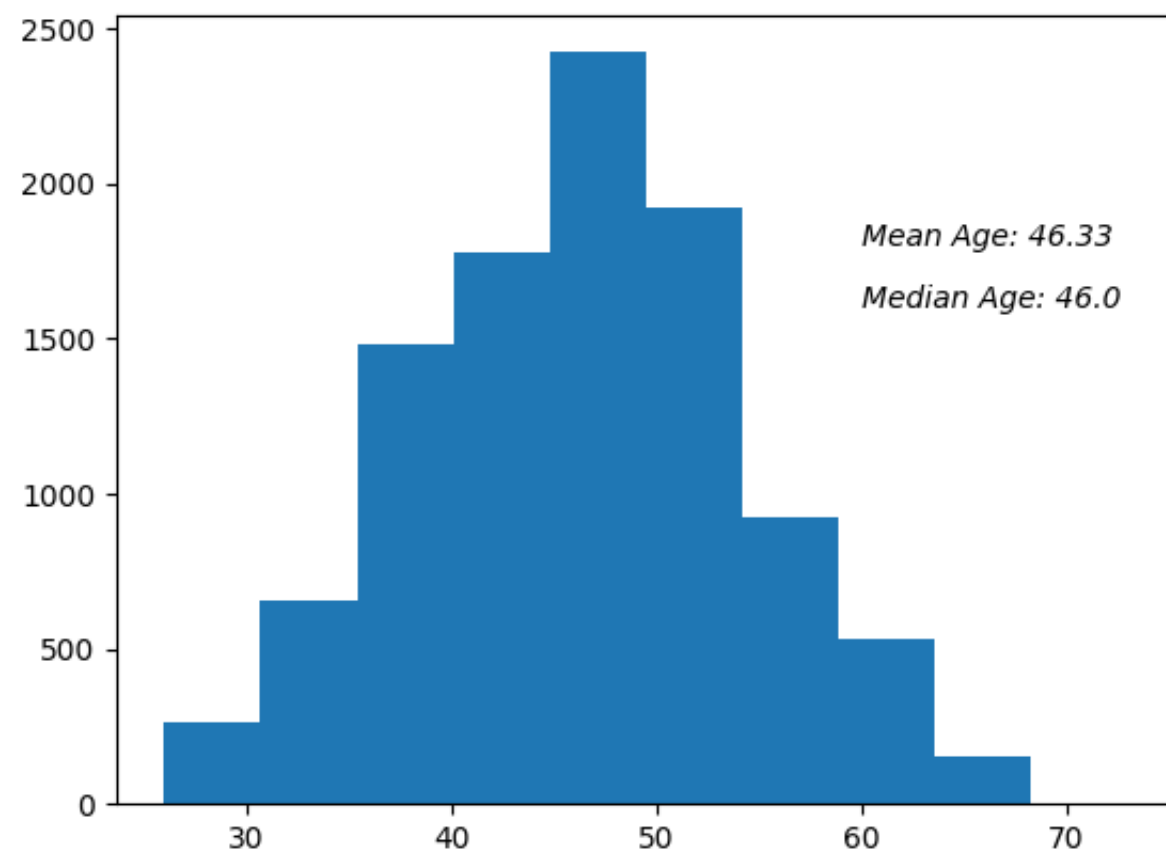


Annotation

Annotation adalah teknik dalam visualisasi data yang digunakan untuk menambahkan teks, panah, atau bentuk lainnya ke dalam grafik atau plot guna memberikan informasi tambahan atau penjelasan tentang elemen-elemen tertentu, seperti titik data, garis, atau area tertentu, sehingga memperjelas interpretasi atau pemahaman grafik.

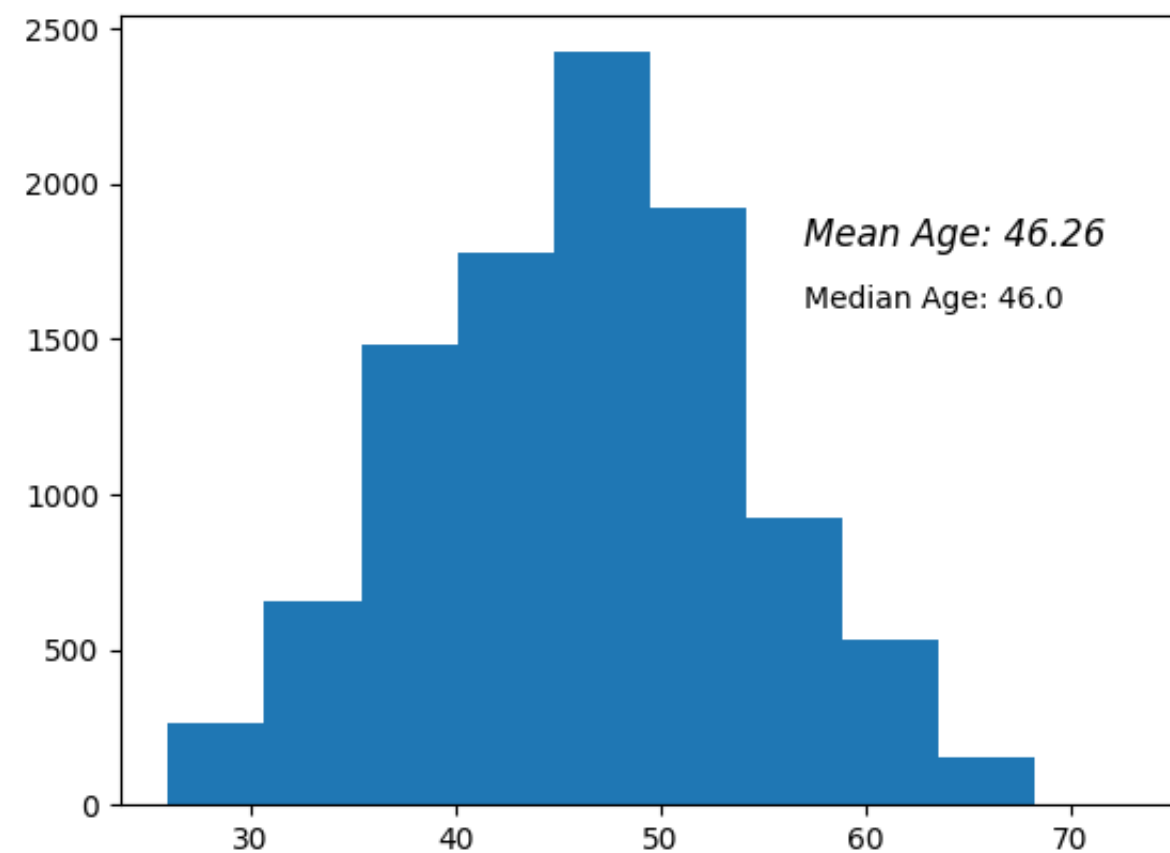
```
In [33]: # Show the distribution of a data and its measure of central tendency
data = df_churning['Customer_Age']
plt.hist(data)

plt.text(60, 1600, "Median Age: " + str(round(data.median(),2)),
        style = 'italic', fontsize=10)
plt.text(60, 1800, "Mean Age: " + str(round(data.mean(),2)),
        style = 'italic', fontsize=10)
plt.show()
```



```
In [34]: fig, ax = plt.subplots()
ax.hist(data)

plt.annotate("Median Age: " + str(round(data1.median(),2)),xy=(57, 1600))
plt.annotate("Mean Age: " + str(round(data1.mean(),2)), xy=(57, 1800), style = 'italic', fontsize=12)
plt.show()
```



Axis

Axis atau sumbu adalah komponen dalam visualisasi data yang menampilkan skala atau rentang nilai dari data yang direpresentasikan dalam grafik atau plot. Sumbu dapat berupa sumbu horizontal (x-axis) yang mewakili variabel independen dan sumbu vertikal (y-axis) yang mewakili variabel dependen dalam hubungan dua dimensi, atau bisa juga melibatkan sumbu lain seperti sumbu z dalam plot tiga dimensi.

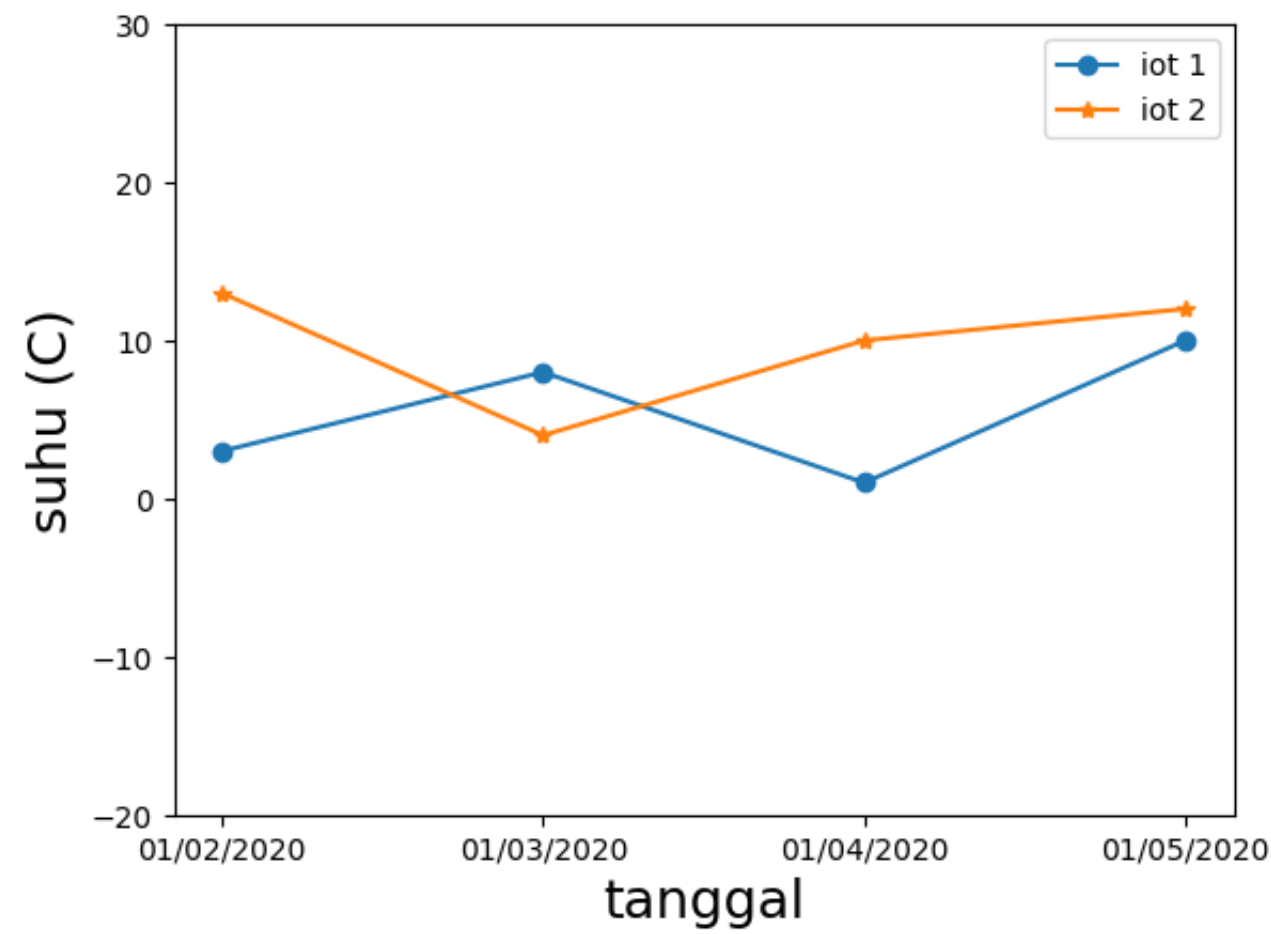
```
In [35]: import matplotlib.pyplot as plt
import numpy as np

#x = np.array([1, 2, 3, 4])
x = np.array(["01/02/2020", "01/03/2020", "01/04/2020", "01/05/2020"])
y1 = np.array([3, 8, 1, 10])
y2 = np.array([13, 4, 10, 12])

plt.plot(x,y1, marker='o', label='iot 1')
plt.plot(x,y2, marker='*', label='iot 2')
plt.legend()

# set label
plt.ylabel('suhu (C)', fontsize=18) # buat label untuk sumbu y
plt.xlabel('tanggal', fontsize=18) # buat label untuk sumbu x

#set axis range
plt.ylim([-20,30])
plt.show()
```

Legend

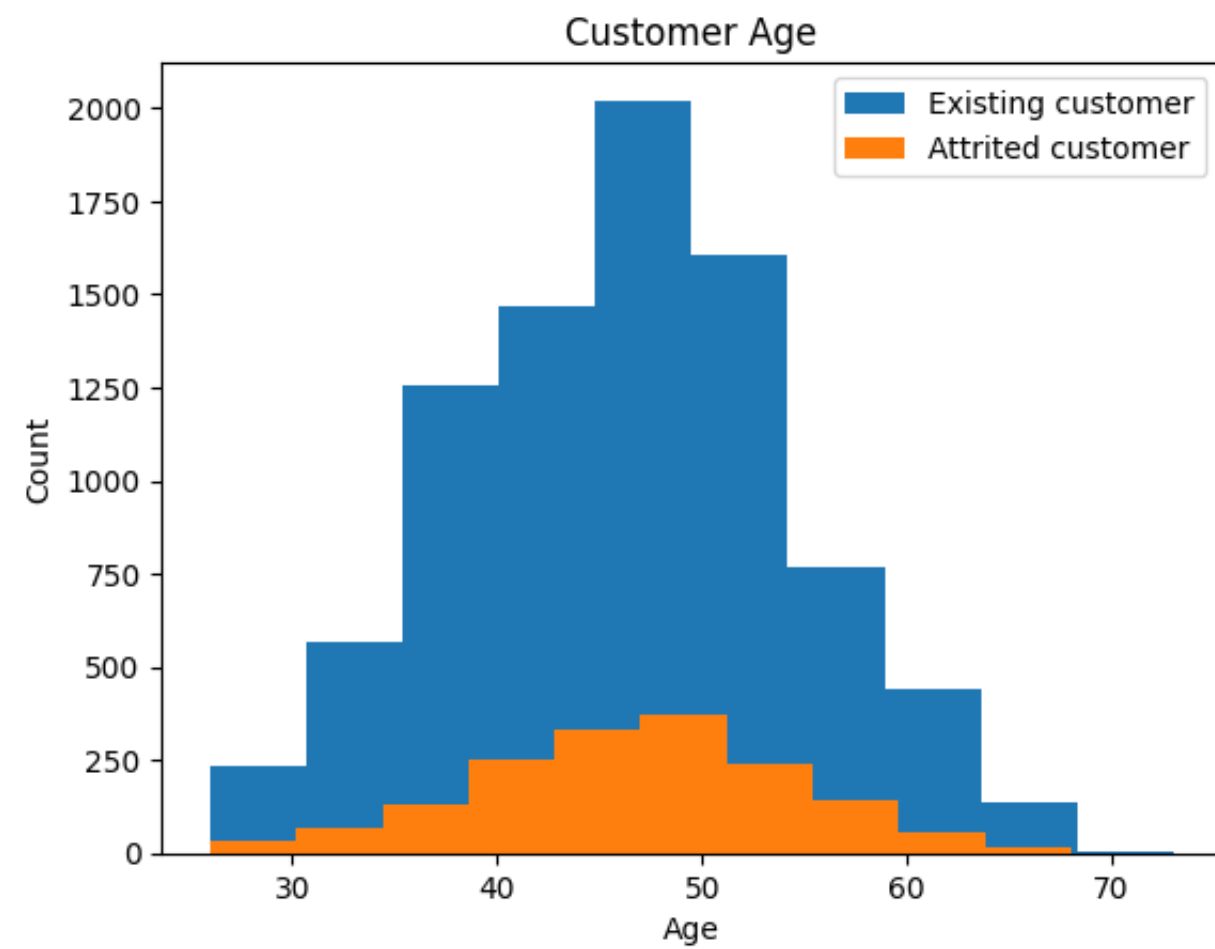
Legend adalah bagian dalam visualisasi data yang menyediakan penjelasan tentang makna atau interpretasi dari elemen-elemen dalam grafik atau plot, seperti warna, bentuk, atau garis. Biasanya terletak di salah satu sudut grafik, legend membantu pembaca untuk mengidentifikasi atau memahami arti dari setiap elemen visual yang digunakan dalam visualisasi data.

```
In [36]: data1 = existing_data['Customer_Age']
data2 = attrited_data['Customer_Age']

plt.hist(data1, label='Existing customer') # pertama, set label dulu
plt.hist(data2, label='Attrited customer')

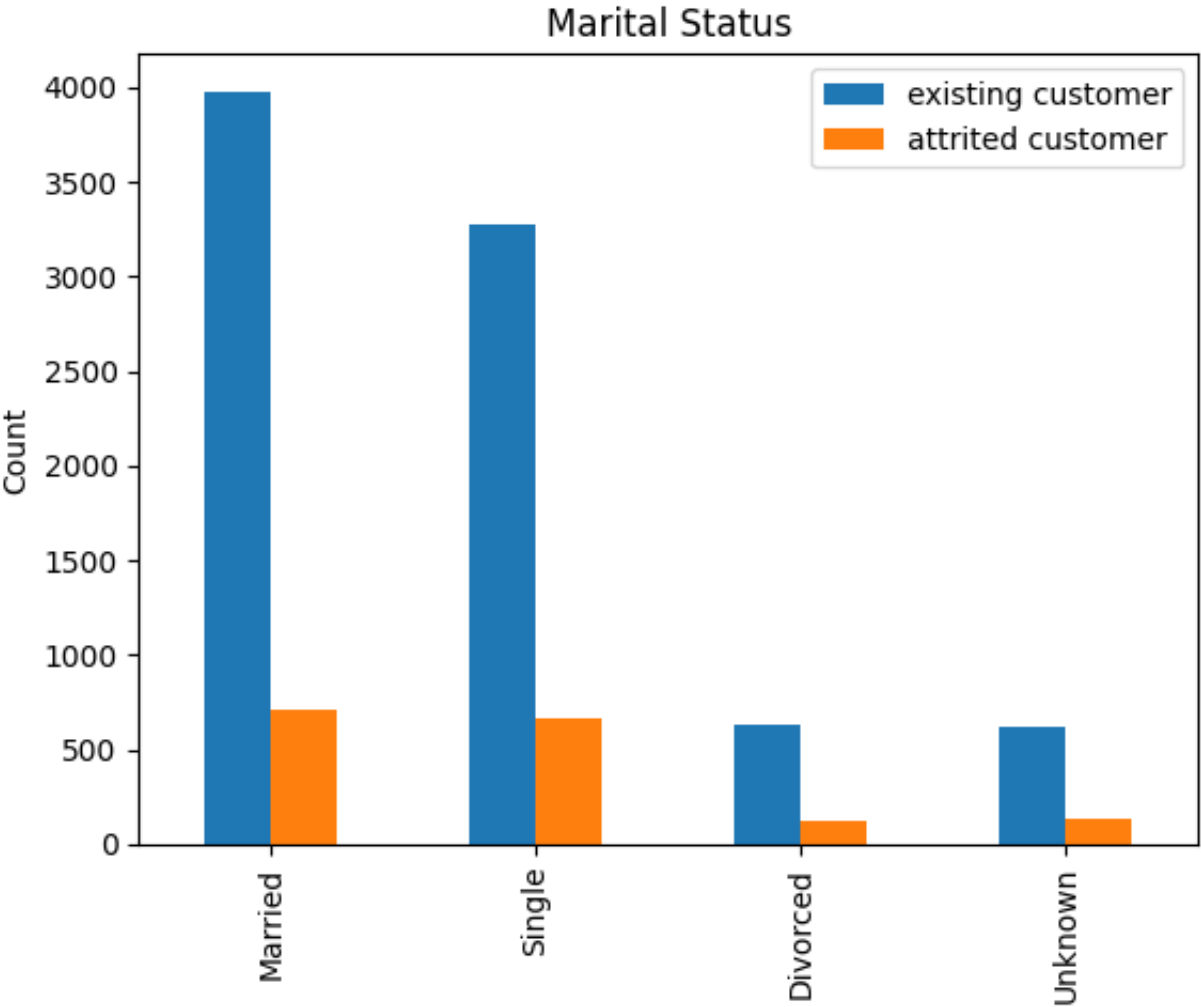
plt.title('Customer Age')
plt.ylabel('Count')
plt.xlabel('Age')

plt.legend() # kedua, panggil fungsi legend
plt.show()
```



```
In [37]: data1 = existing_data['Marital_Status'].value_counts()
data2 = attrited_data['Marital_Status'].value_counts()
df_new = pd.concat([data1, data2], keys=['existing customer', 'attrited customer'], axis=1)

#df_new.plot.bar() #bisa pakai cara 1
df_new.plot(kind='bar') # bisa pakai cara 2
plt.title('Marital Status')
plt.ylabel('Count')
#plt.legend() # sudah otomatis, tidak perlu dipanggil.
plt.show()
```



Summary

Penjelasan penggunaan grafik dalam visualisasi data sebagai berikut:

Distribusi (Distribution):

Histogram: Digunakan untuk menampilkan distribusi frekuensi atau probabilitas dari data numerik. Setiap batang dalam histogram mewakili interval data, dan tinggi batang menunjukkan frekuensi atau probabilitas kejadian di interval tersebut.

Scatter plot: Digunakan untuk menampilkan distribusi data titik dan menampilkan pola atau hubungan antara dua variabel numerik. Setiap titik dalam scatter plot mewakili satu pengamatan dan posisinya ditentukan oleh nilai dari dua variabel yang berkaitan.

Hubungan (Relationship):

Scatter plot: Digunakan untuk mengeksplorasi hubungan antara dua variabel numerik. Dengan menggambar titik-titik pada bidang kartesian, scatter plot membantu dalam menemukan pola atau korelasi antara variabel-variabel tersebut.

Perbandingan (Comparison):

Bar plot: Digunakan untuk membandingkan nilai-nilai kategori dalam satu variabel. Setiap batang dalam bar plot mewakili satu kategori dan tingginya menunjukkan nilai atau frekuensi kategori tersebut.

Line plot: Digunakan untuk membandingkan tren atau perubahan nilai variabel numerik dari waktu ke waktu. Garis yang menghubungkan titik-titik data membantu dalam menyoroti pola atau tren dari data tersebut.

Komposisi (Composition)

Pie chart: Digunakan untuk menunjukkan komposisi relatif dari bagian-bagian. Setiap "slice" dari pie chart mewakili proporsi atau persentase dari keseluruhan.

Stacked bar chart: Digunakan untuk menunjukkan komposisi variabel dalam satu kategori. Setiap batang dalam stacked bar chart dibagi menjadi segmen-segmen yang mewakili kontribusi dari variabel-variabel tersebut pada kategori tersebut.